Robot Perception and Learning

Real-world RL

Tsung-Wei Ke

Fall 2026



Updated Schedule

Date	Course	Announcements	Student Presentation
9/7	Introduction		
9/16	Robot Kinematics, Motion Planning		
9/21	Motion Planning, Feedback Control (PID)	Team up for HW1 (9/28)	
10/5	Feedback Control (Impedance / Admittance Control), Optimal Control	HW1 release	
10/12	Imitation Learning		
10/19	Introduction of Reinforcement Learning, Dynamic Programming, Monte Carlo, Temporal Difference		
10/27	Deep RL (Q learning, Policy Gradient)	Withdrawal deadline (10/30)	
11/2	~~Offline RL,~~ Challenges of Real-world RL	HW2 release	Final project proposal
11/9	Model-based RL		Chat with TW

Tentative Schedule

Date	Course	Announcements	Student Presentation
11/16	Zero-shot RL, Intelligent exploration		
11/23	Visual Imitation Learning, Inverse RL, Reward Learning		
11/30	Sim2real transfer, Real2Sim2Real Learning, Adaptation		Chat with TW
12/7	Special topic: Agentic Robotics		
12/14	Special topic: Bimanual Dexterous Manipulation and Mobile Manipulation		
12/28			Poster session

IL is Incapable of Generalization

Succeed in the real world



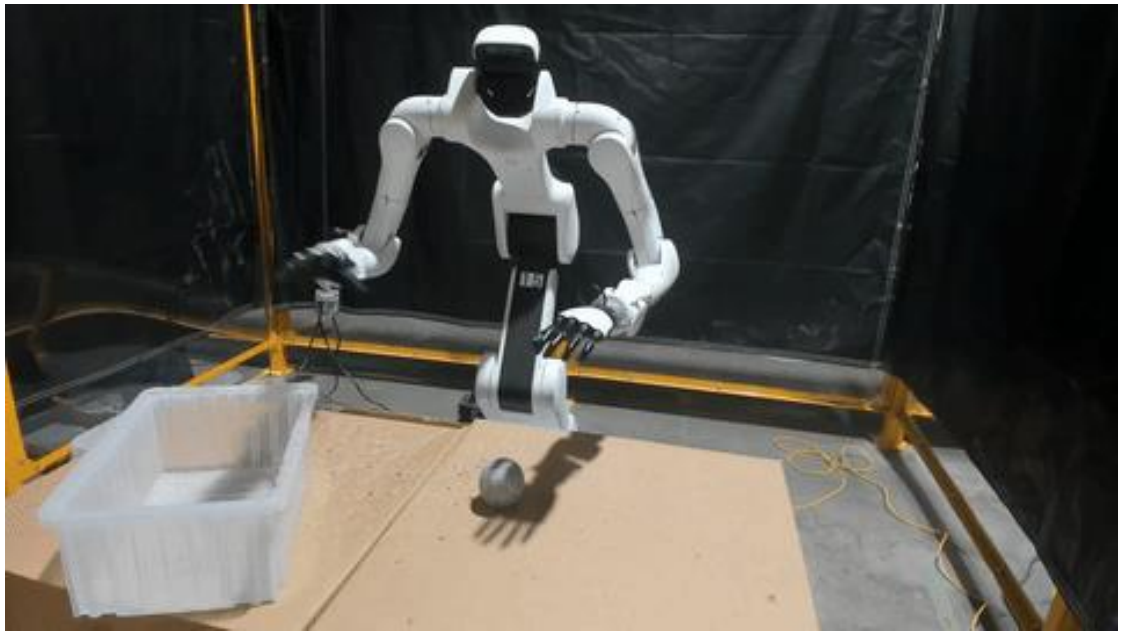
OpenVLA: An Open-Source Vision-Language-Action Model

Fail in the digital twin

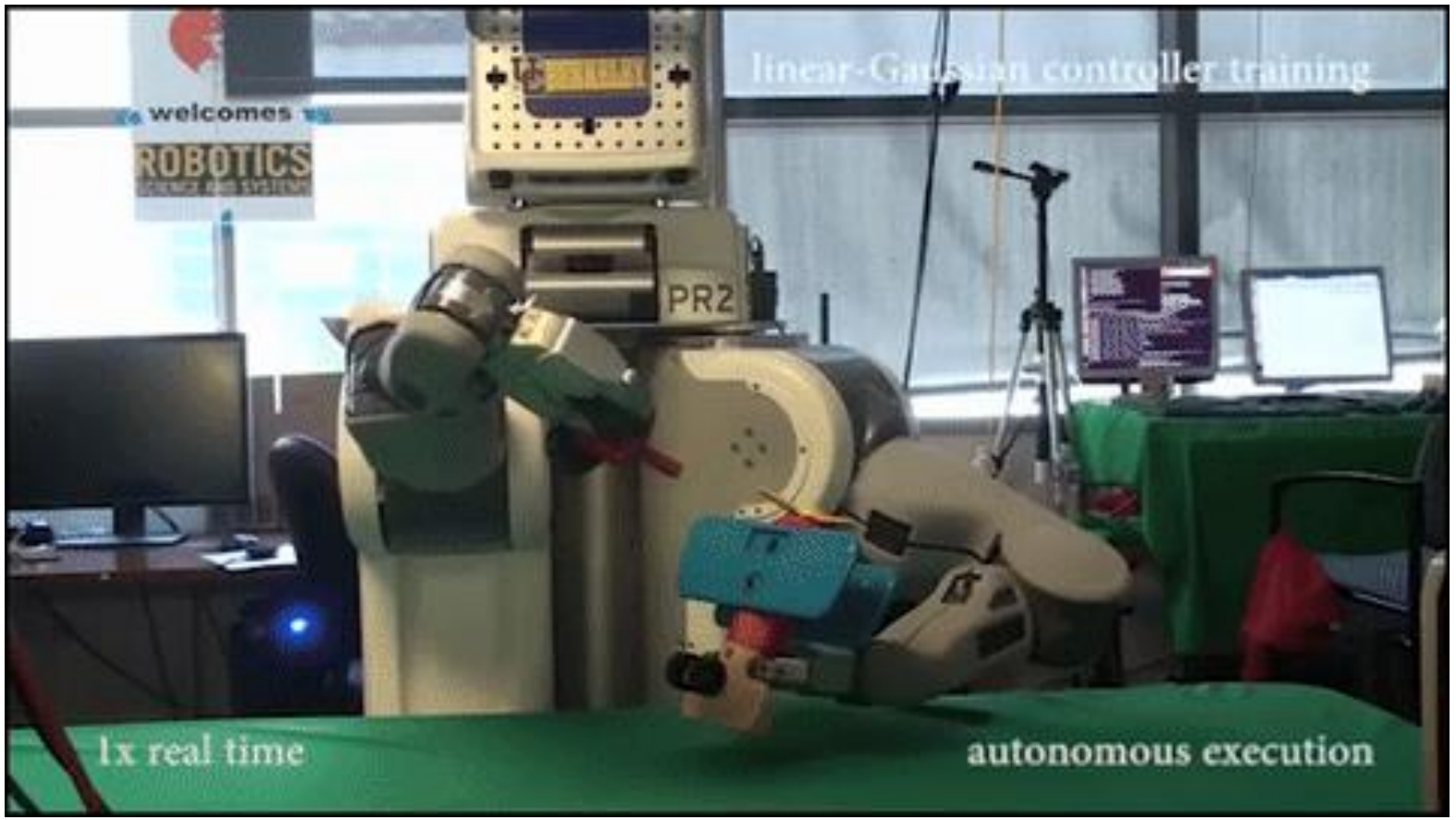


IL is Incapable of Generalization





Idea: Enhance Generalization of IL Policy with RL



https://www.youtube.com/watch?v=JeVppkoloXs&t=11s

Step 1: Implement the Actor and Critic Network

PPO algorithm

Input: initial policy parameters θ_0, initial KL penalty β_0, target KL-divergence δ
for $k = 0, 1, 2, \ldots$ do
  Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$
  Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm
  Compute policy update

$$\theta_{k+1} = \arg\max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \beta_k \bar{D}_{KL}(\theta||\theta_k)$$

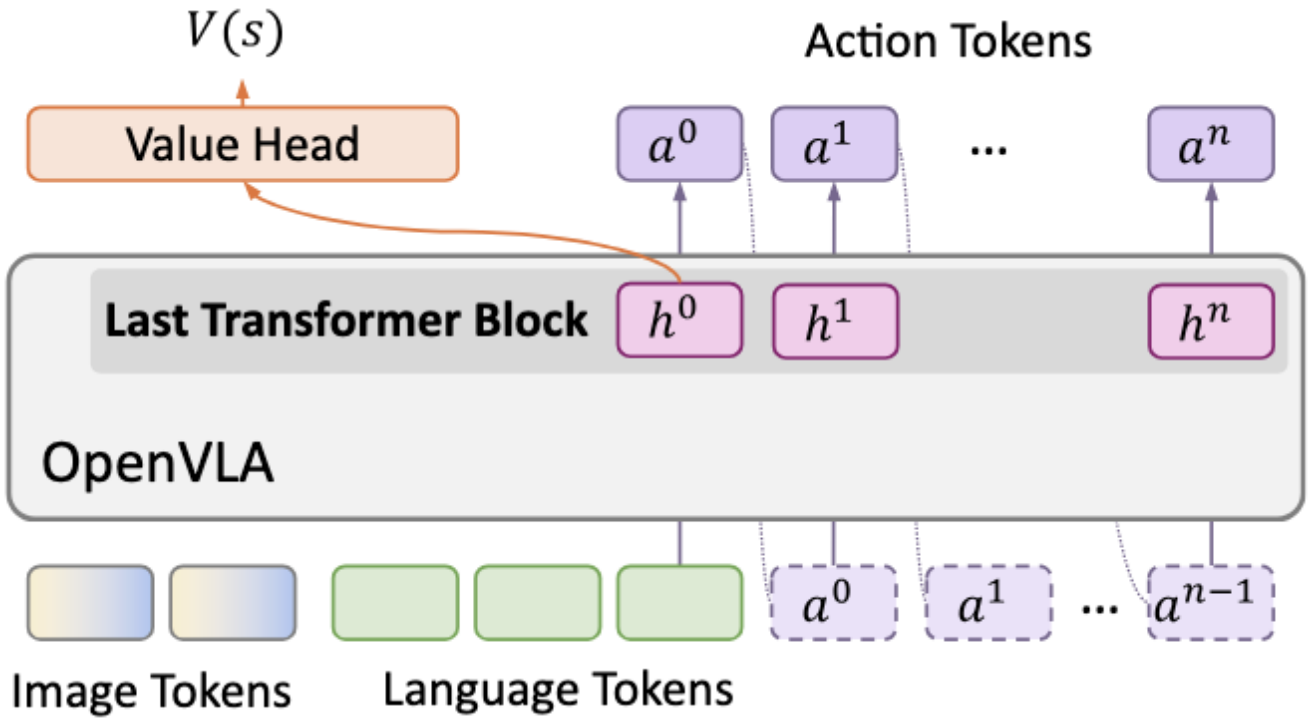
  by taking K steps of minibatch SGD (via Adam)
  if $\bar{D}_{KL}(\theta_{k+1}||\theta_k) \geq 1.5\delta$ then
    $\beta_{k+1} = 2\beta_k$
  else if $\bar{D}_{KL}(\theta_{k+1}||\theta_k) \leq \delta/1.5$ then
    $\beta_{k+1} = \beta_k/2$
  end if
end for

Collect trajectories and estimate $A(s, a)$

Update the policy

Adaptively tune the penalty coefficient

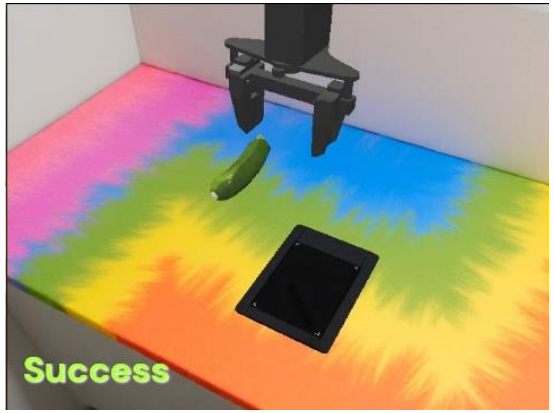
Don't use second order approximation for Kl which is expensive, use standard gradient descent



Step 2: Create Diverse Environments



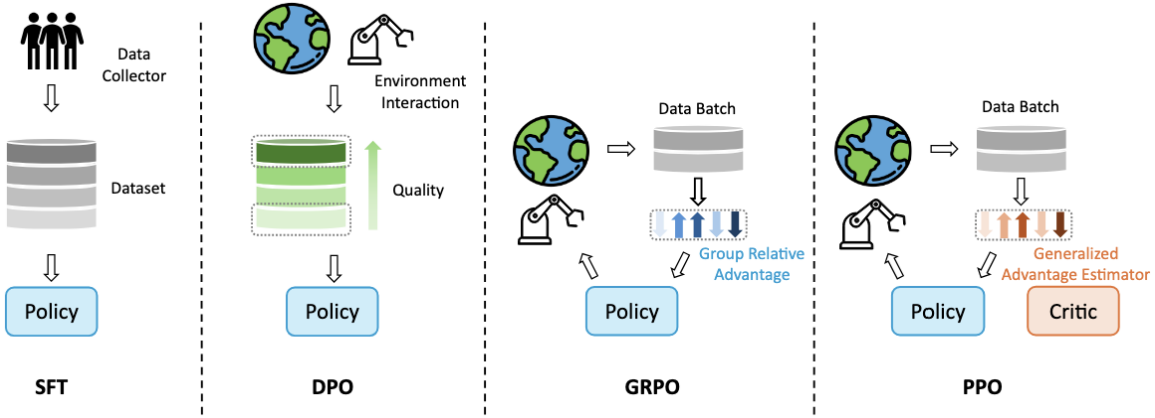


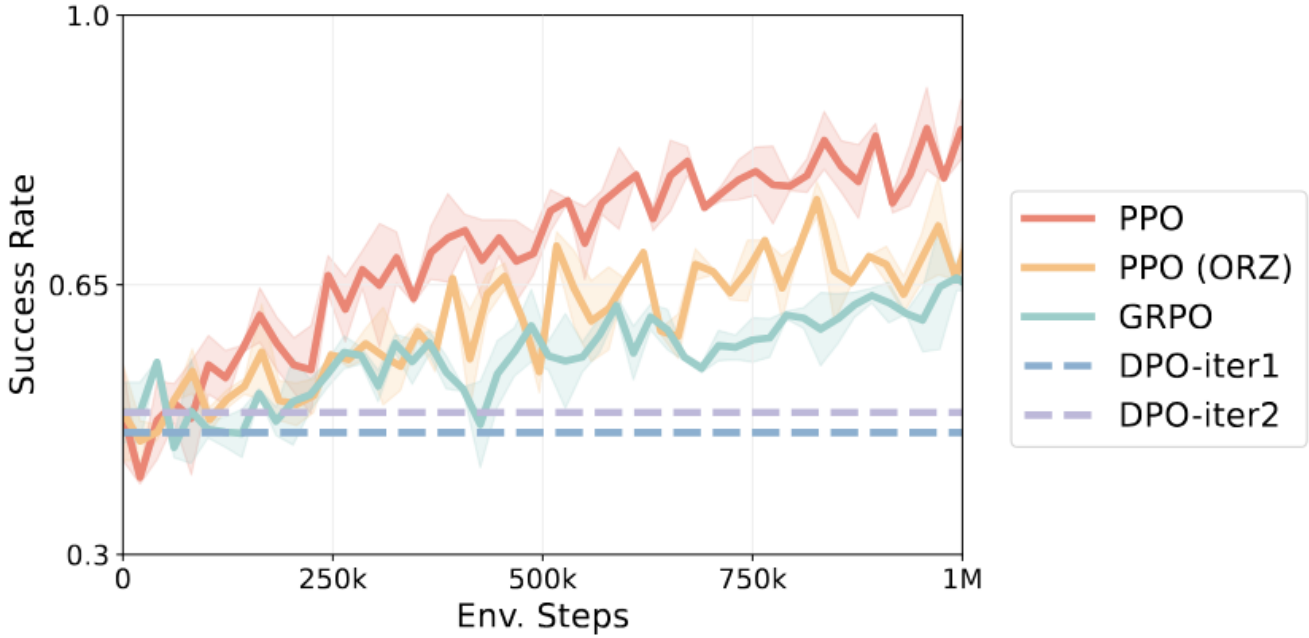




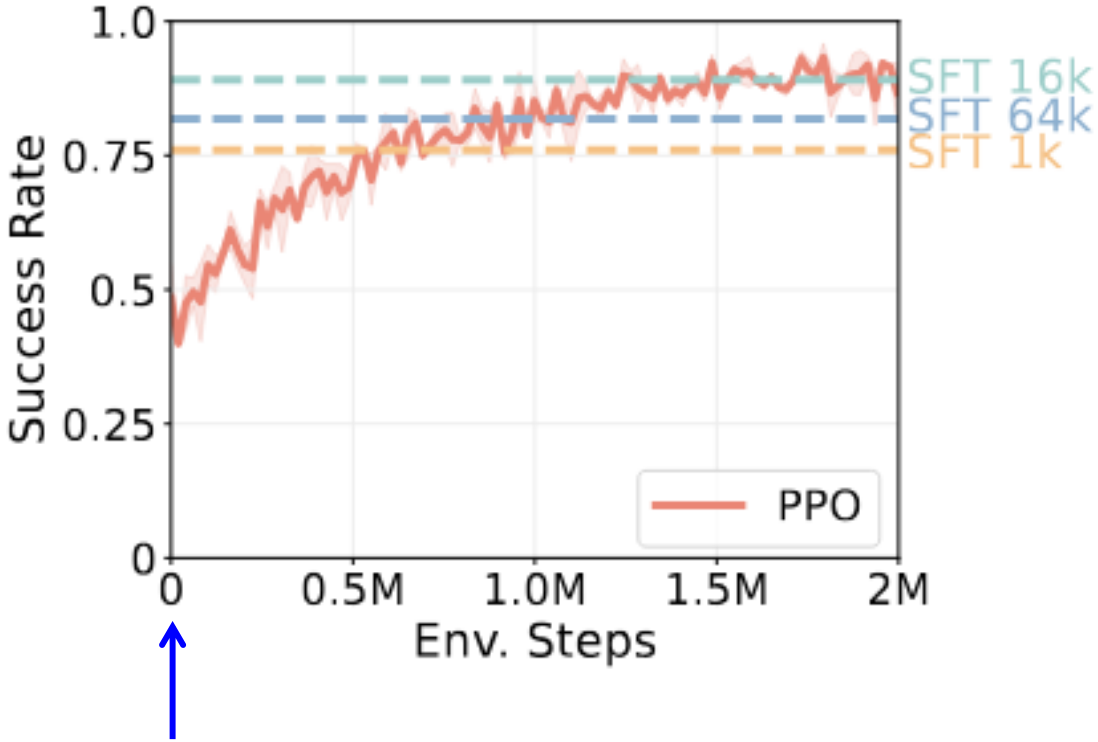


Step 3: Fine-tune IL Policy with PPO

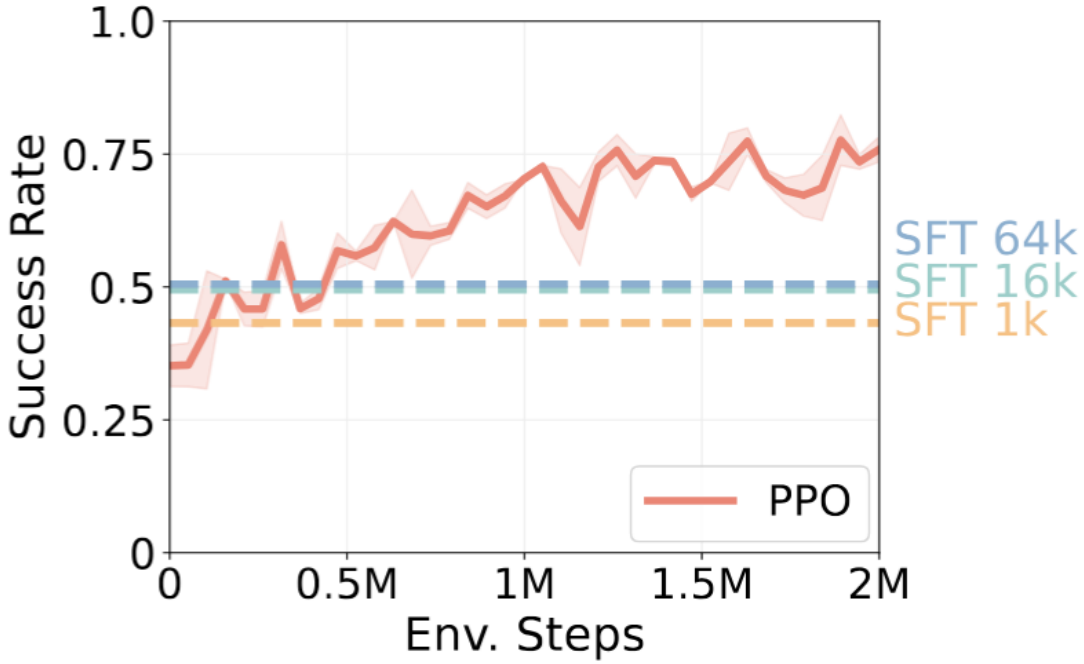




Results: Policy Generalizes to the Current Simulation Environment



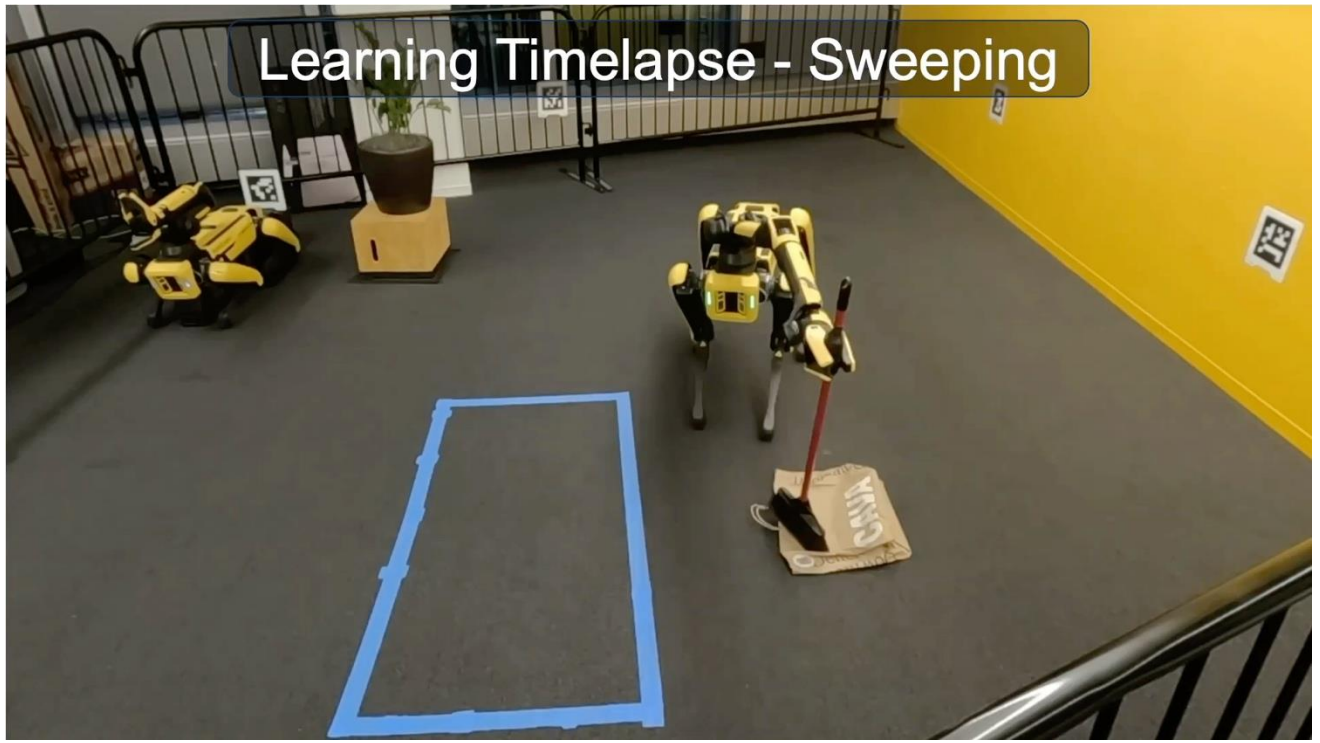
Before RL finetuning



Can We Do RL in the Real World?



MT-Opt: Continuous Multi-Task Robotic Reinforcement Learning at Scale. Kalashnikov et al.



Continuously Improving Mobile Manipulation with Autonomous Real-World RL. Mendonca et al.

Problems of Fine-tuning VLAs via RL in the Real-World

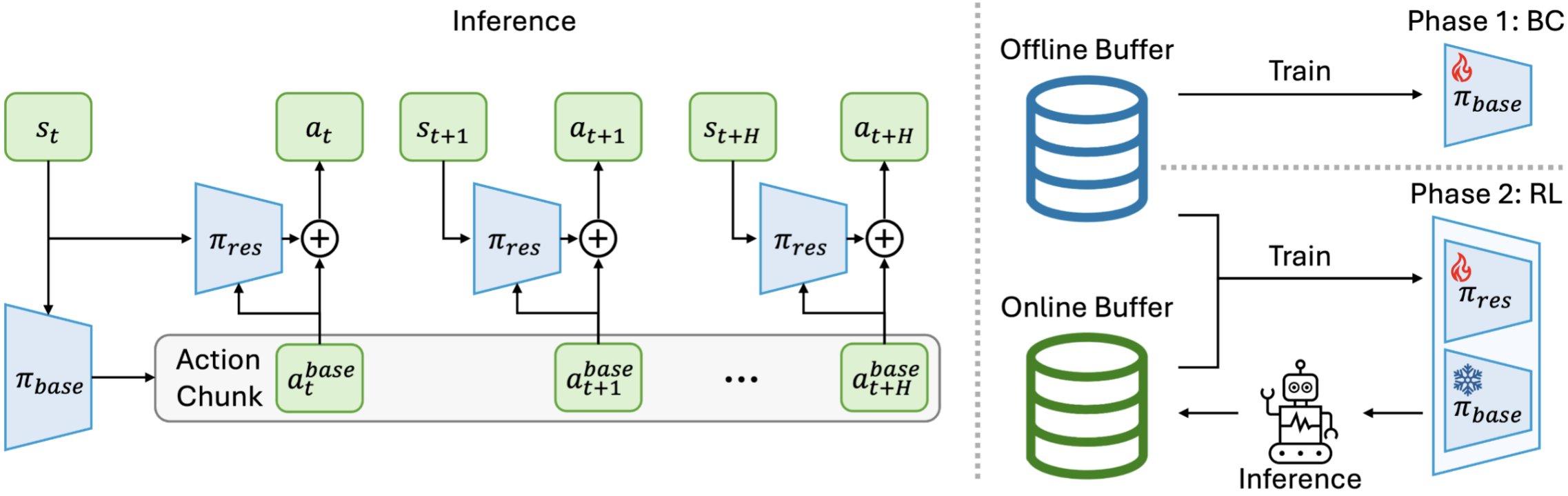
- Heavy computational overhead of fine-tuning the entire VLA

Computational Overhead of Fine-tuning the Entire VLA

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	**69.7 ± 7.2 %**	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	**68.2 ± 7.5%**	**97.6**	**59.7 GB**
rank=64	**68.2 ± 7.8%**	195.2	60.5 GB

OpenVLA: An Open-Source Vision-Language-Action Model. Kim et al.

Idea: Adapt the VLA with a Small Residual Policy



Fine-tuning with Residual RL

Algorithm 1 ResFiT: Residual fine-tuning w/ Off-Policy RL

Input: pre-trained base π_{b}, dataset of demos $\mathcal{D}_{\text{offline}}$
Initialize: residual policy π_θ, Q ensemble $Q_{\phi_1}, \dots Q_{\phi_N}$, vision encoder f_ω, empty buffer $\mathcal{D}_{\text{online}}$
Initialize: set target networks $\theta' \leftarrow \theta$, $\phi'_i \leftarrow \phi_i$

for step $= 1, 2, \dots,$ `buffer_warmup_steps` **do**
  Sample noise $\epsilon_t \sim \mathcal{U}(-\text{noise_scale}, \text{noise_scale})$
  Step env with $a_t = \epsilon_t + a_t^{\text{b}}$ where $a_t^{\text{b}} \sim \pi_{\text{b}}(s_t)$
  Observe next state s_{t+1}, reward r_t, done flag d_t
  Add transition $(s_t, a_t^{\text{b}}, a_t, s_{t+1}, a_{t+1}^{b}, r_t, d_t)$ to $\mathcal{D}_{\text{online}}$
end for
repeat
  $a_t^{\text{b}} \sim \pi_{\text{b}}(s_t)$ and $a_t^{\text{r}} \sim \pi_\theta(s_t, a_t^{\text{b}})$
  Step env with $a_t = a_t^{\text{b}} + a_t^{\text{r}}$
  Add $(s_t, a_t^{\text{b}}, a_t, s_{t+1}, a_{t+1}^{b}, r_t, d_t)$ to $\mathcal{D}_{\text{online}}$
  Repeat UTD times:
    Sample batch B evenly from $\mathcal{D}_{\text{offline}}$, $\mathcal{D}_{\text{online}}$
    Update critic using B:
      $a_{t+1}^{\text{r}} \sim \pi_{\theta'}(s_{t+1}, a_{t+1}^{\text{b}})$
      Compute $a_{t+1} = a_{t+1}^{\text{b}} + a_{t+1}^{\text{r}}$
      $y = r_t + (1 - d_t) * \gamma * \min_{\text{subset(i)}} Q_{\phi'_i}(s_{t+1}, a_{t+1})$
      Update ϕ_i, ω to minimize $\text{MSE}(Q_{\phi_i}(s_t, a_t), y)$
      Update critic targets $\phi'_i \leftarrow \rho\phi'_i + (1 - \rho)\phi_i$
  Update actor using latest B:
    Update θ to maximize:
      $\frac{1}{N} \sum_{i=1}^{N} Q_{\phi_i}(s_t, \pi_\theta(s_t, a_t^{\text{b}}) + a_t^{\text{b}})$
    Update actor target $\theta' \leftarrow \rho\theta' + (1 - \rho)\theta$
until convergence

Warm-up replay buffer

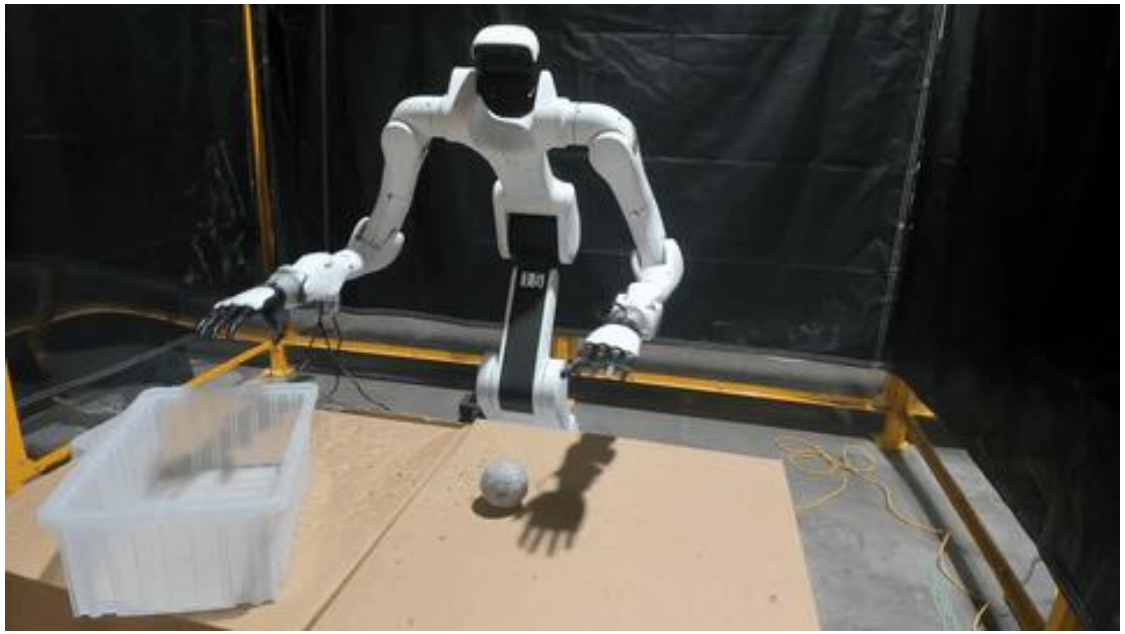
Fit the value network

Fit the policy network

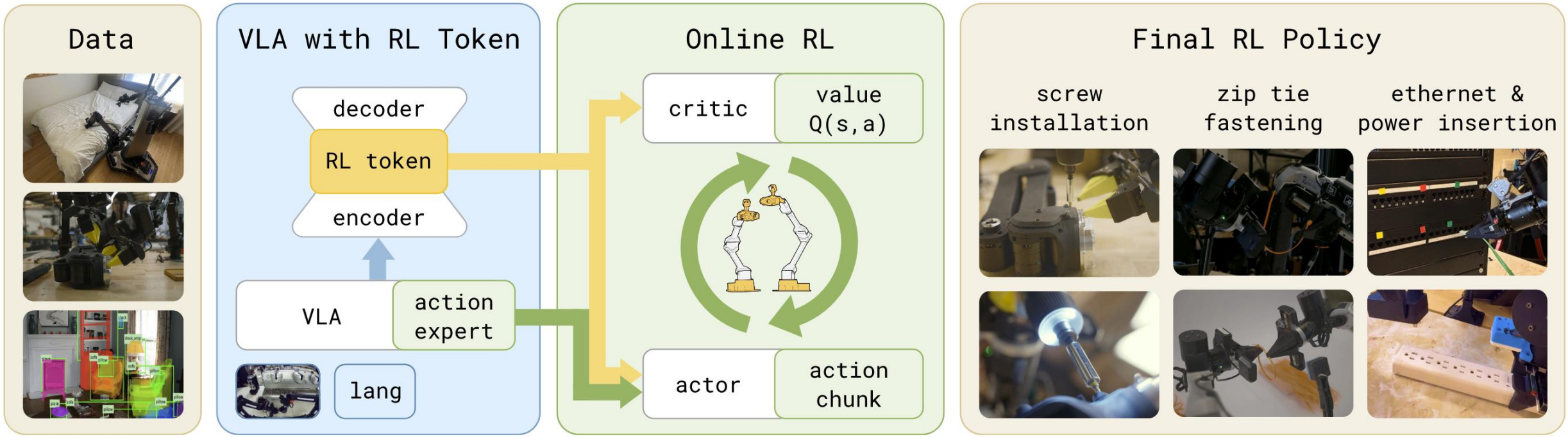
Residual Off-Policy RL for Finetuning Behavior Cloning Policies. Ankile et al.

Real-world RL Enhances the Policy

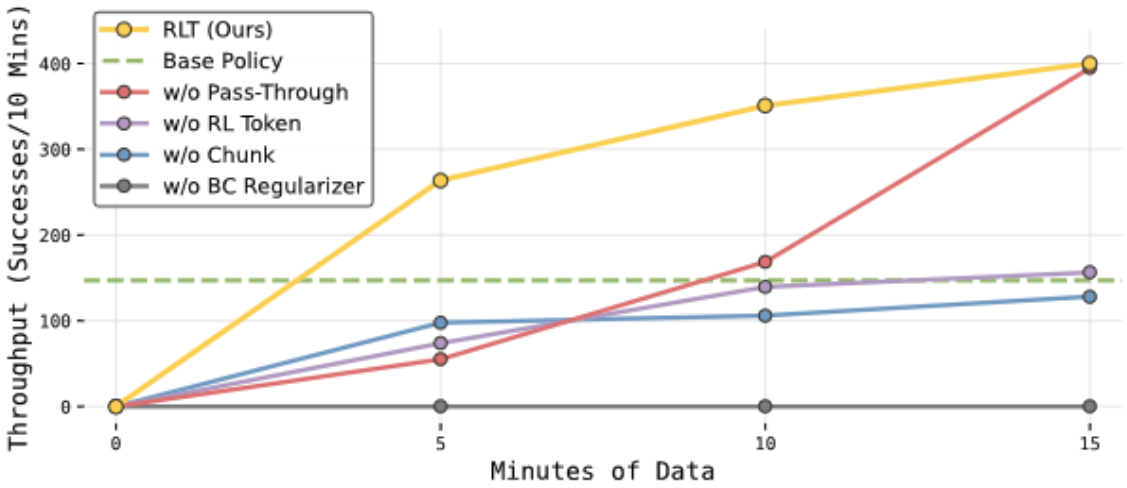


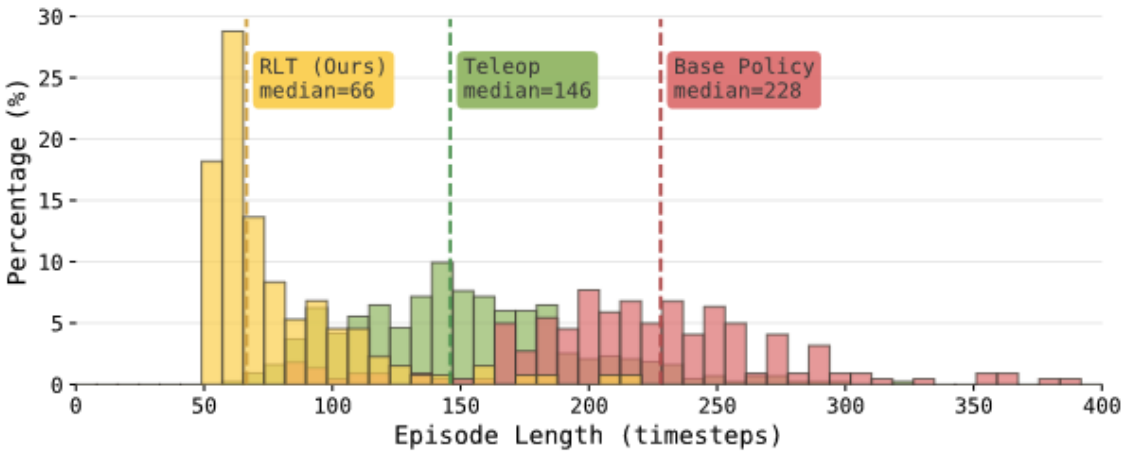


Residual RL in Cooler Forms



Residual RL in Cooler Forms

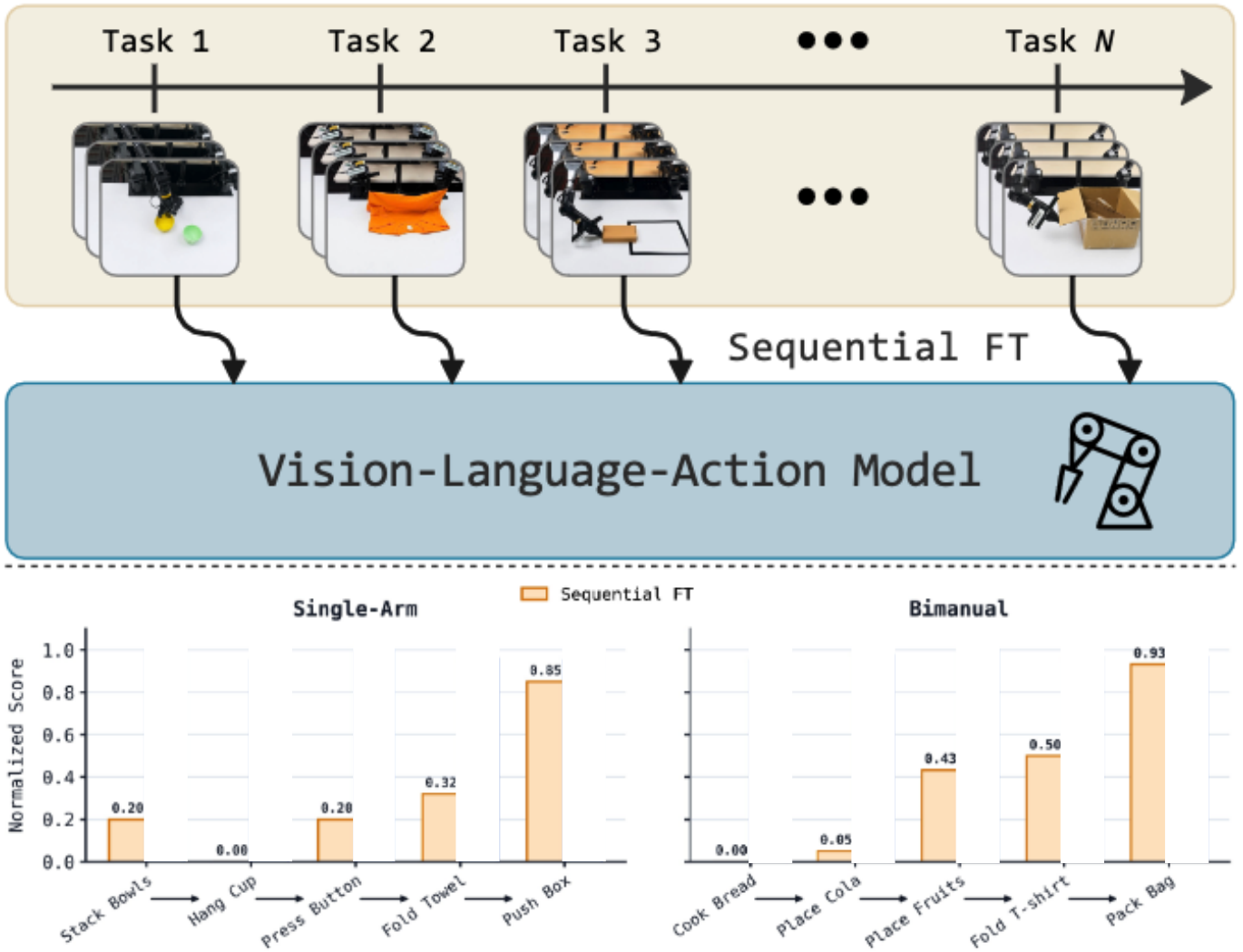




Problems of Fine-tuning VLAs via RL in the Real-World

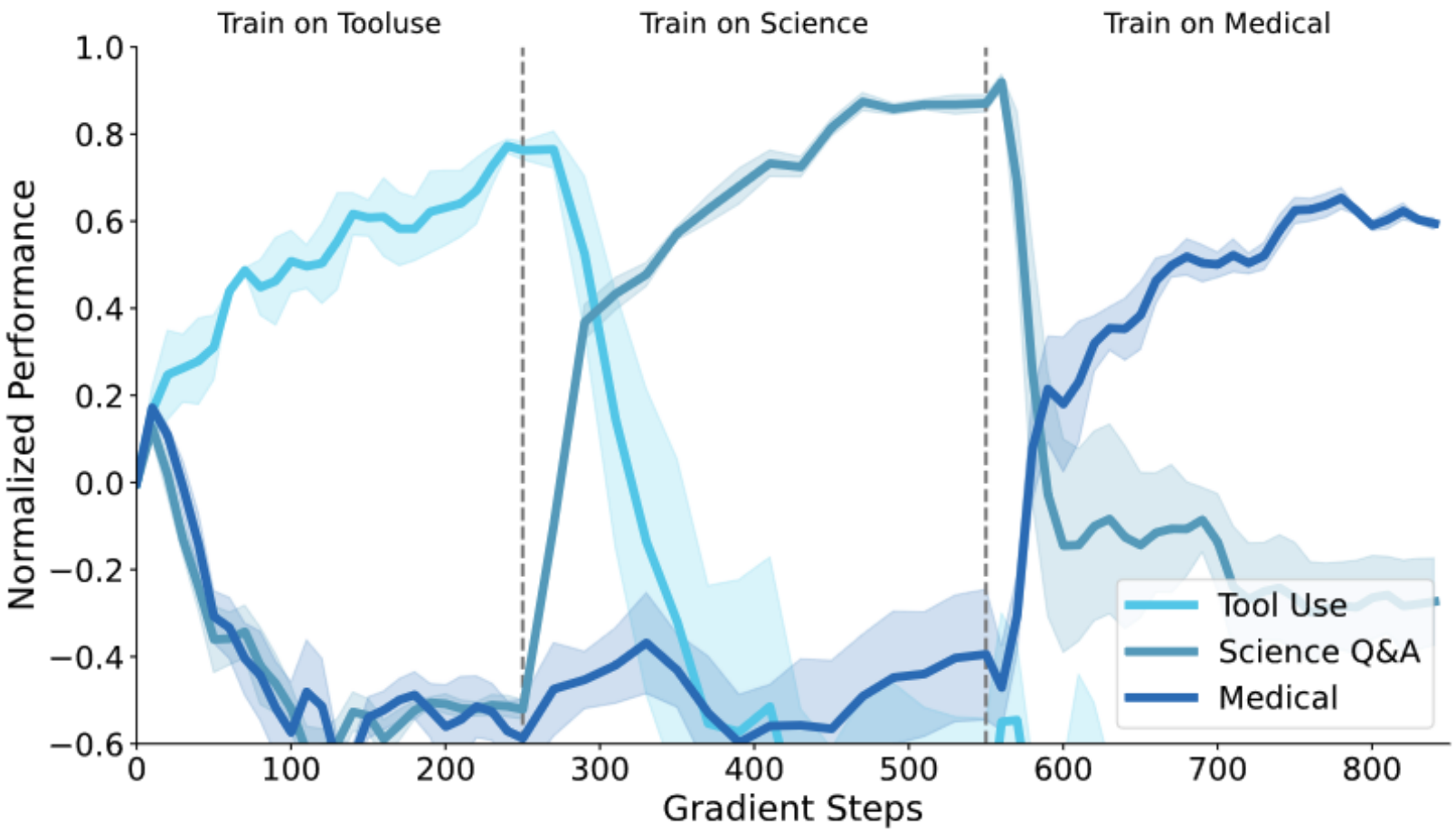
- Heavy computational overhead of fine-tuning the entire VLA
- Catastrophic forgetting issue after fine-tuning

Models Forget Old Skills after Learning New Skills

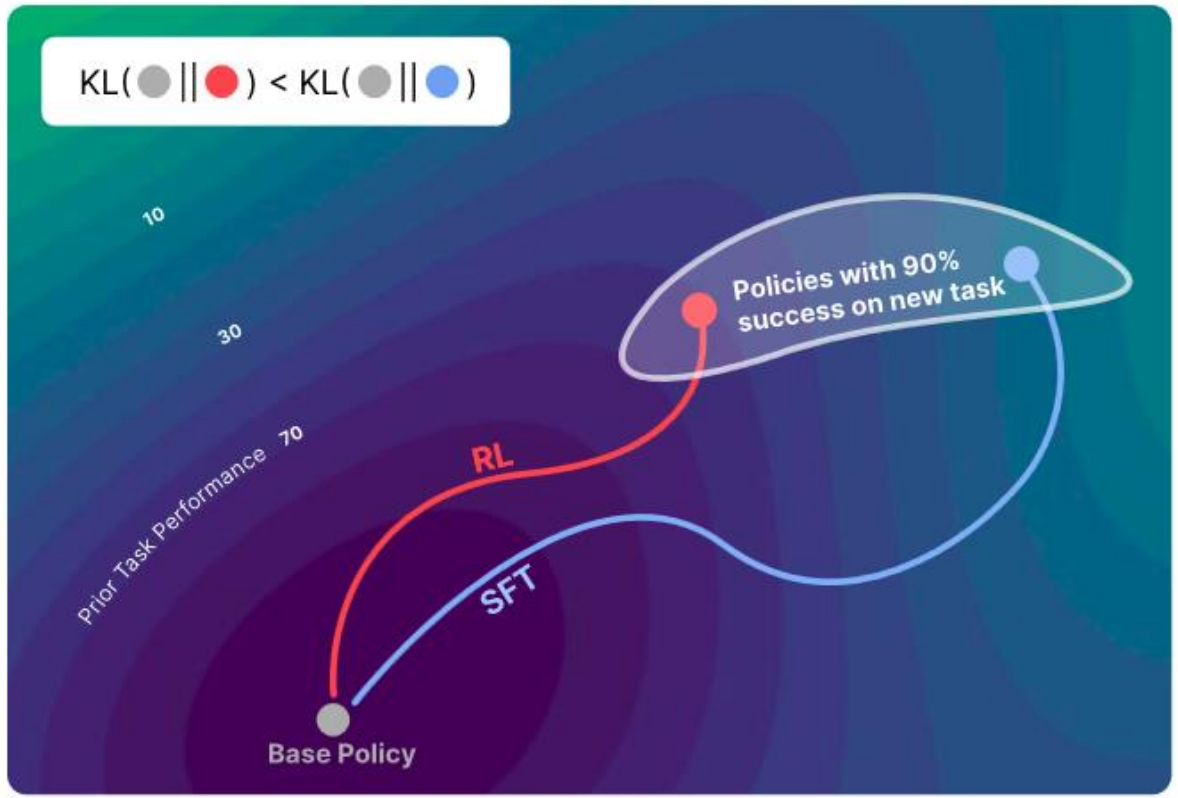


Can Vision-Language-Action Models Learn from Real-World Data Continually without Forgetting? Zhu et al

Models Forget Old Skills after Learning New Skills



RL Fine-tuning Forgets Less than Supervised Fine-tuning





Why Does RL Fine-tuning Forgets Less?

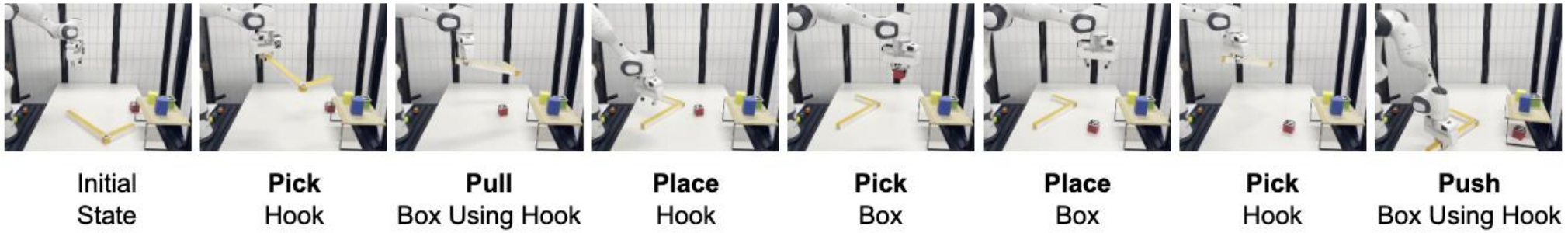
- Remember trust-region RL's objective

$$\max_{\pi} \mathrm{J}(\pi) - \mathrm{J}(\pi_{\mathrm{old}}) \geq \mathcal{L}_{\pi_{old}}(\pi) - \mathrm{C}\sqrt{\underset{\mathrm{s}\sim d^{\pi_{old}}}{E}[D_{KL}(\pi|\pi_{old})[\mathrm{s}]]}$$

This regularization avoids deviation from old policy distribution

Any better idea?

Idea 1: Joint training on demonstrations of old tasks and trajectories of new tasks



Initial State | **Pick** Hook | **Pull** Box Using Hook | **Place** Hook | **Pick** Box | **Place** Box | **Pick** Hook | **Push** Box Using Hook

$o_1^1, a_1^1, o_2^1, a_2^1, \ldots, o_T^1, a_T^1$
$o_1^2, a_1^2, o_2^2, a_2^2, \ldots, o_T^2, a_T^2$
$\vdots$
$o_1^N, a_1^N, o_2^N, a_2^N, \ldots, o_T^N, a_T^N$

Demonstration dataset

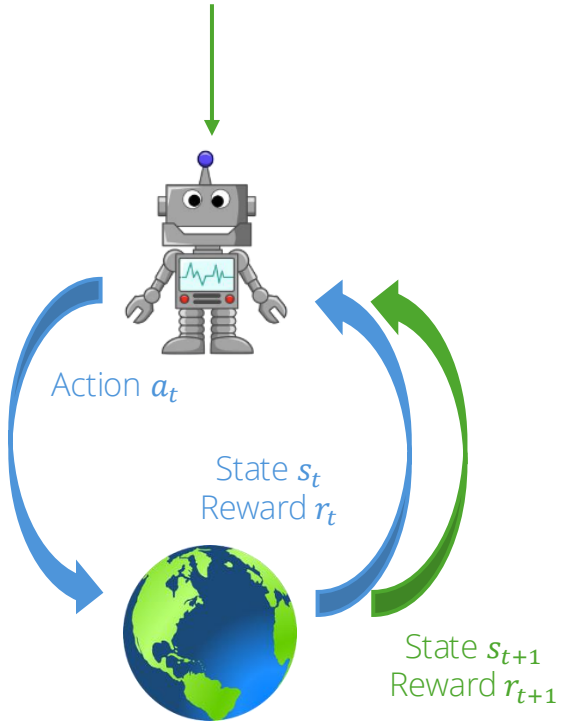
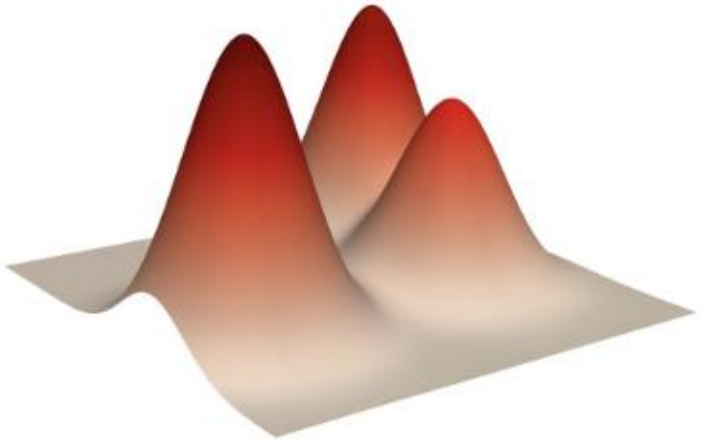


Image source: Mishra et al

Idea 2: Learning New Tasks with Fixed Action Distribution

- Intuition: Pre-trained robot policies have learned good and broad action distributions. Policies fail in new tasks / environments because they don't select right actions.

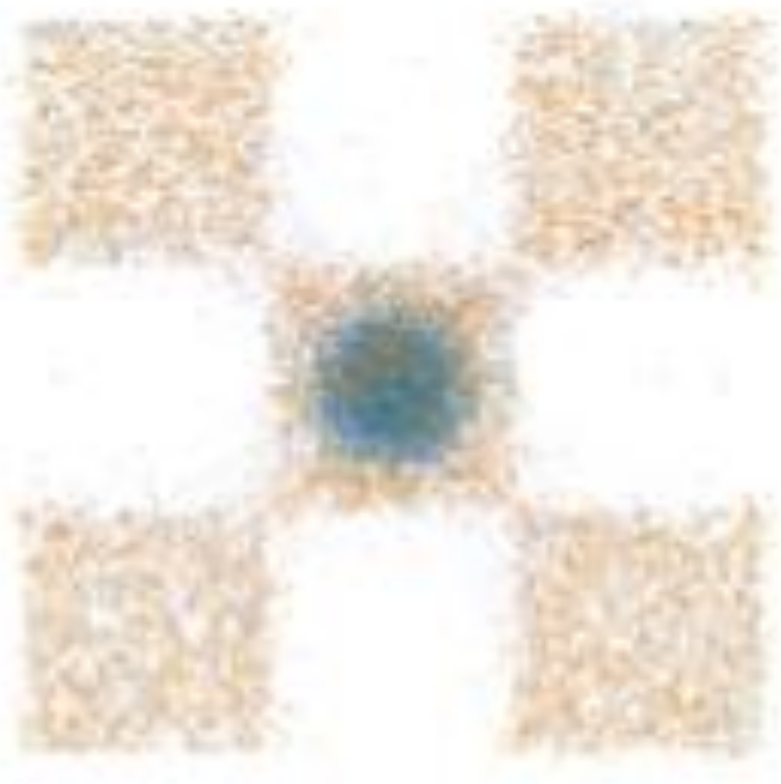




Video source: DROID dataset

Idea 2: Learning New Tasks with Fixed Action Distribution

- Intuition: the output of flow matching / diffusion policies depends on sampled conditioning noise



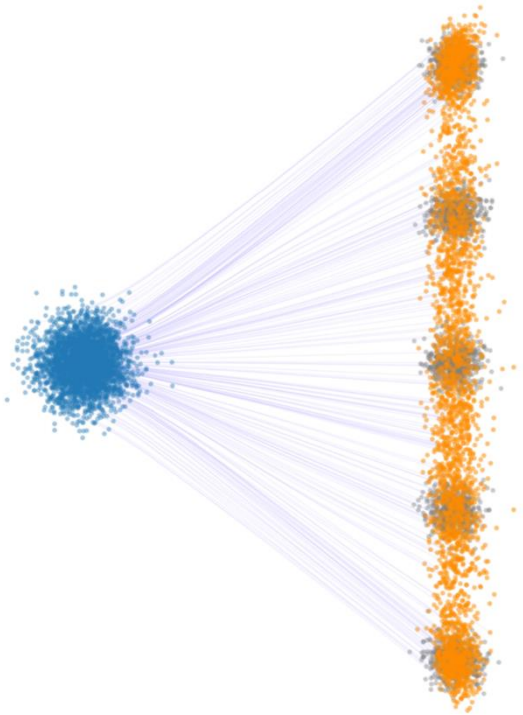
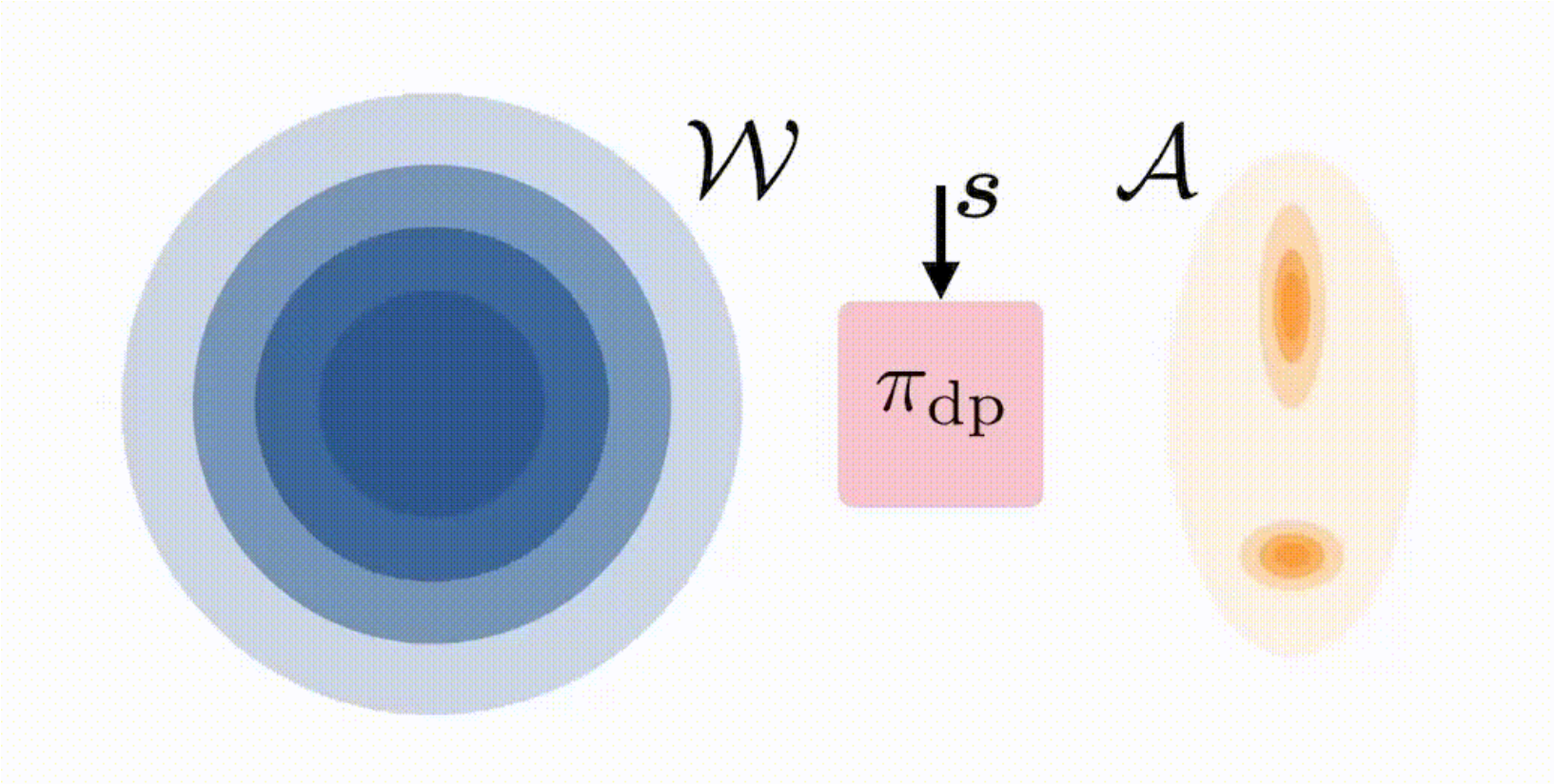


Image source: Chiang et al

Idea 2: Learning New Tasks with Fixed Action Distribution

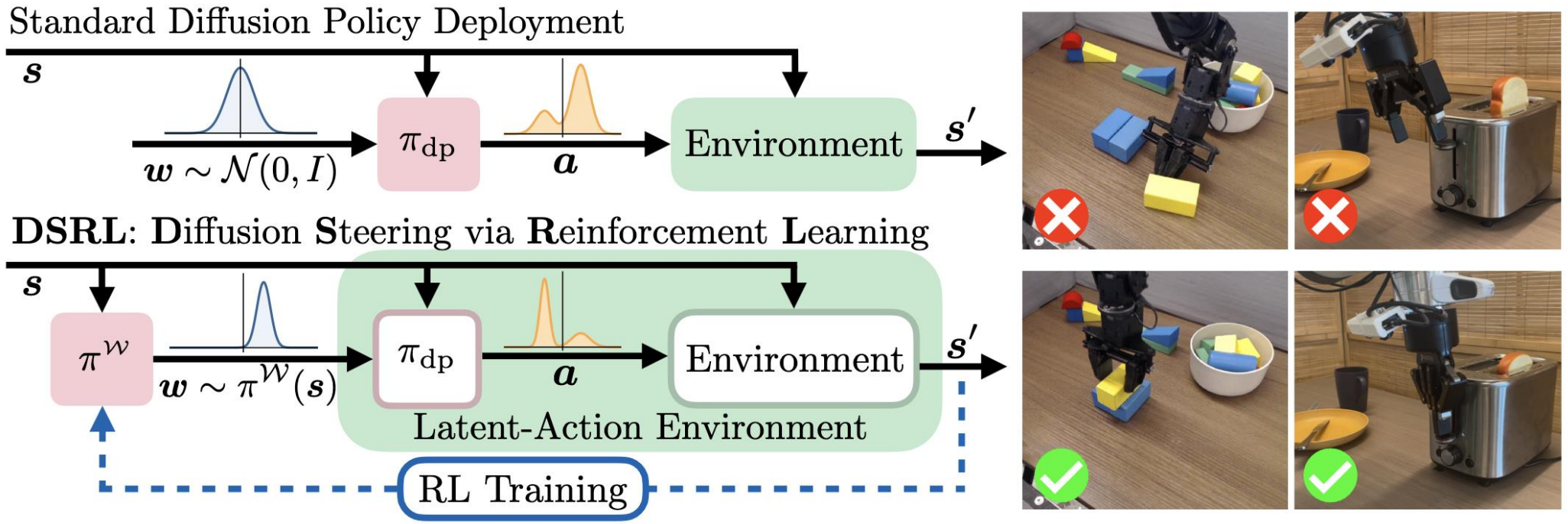
- Idea: train a noise predictor to steer diffusion robot policies



Steering Your Diffusion Policy with Latent Space Reinforcement Learning. Wagenmaker et al.

Idea 2: Learning New Tasks with Fixed Action Distribution

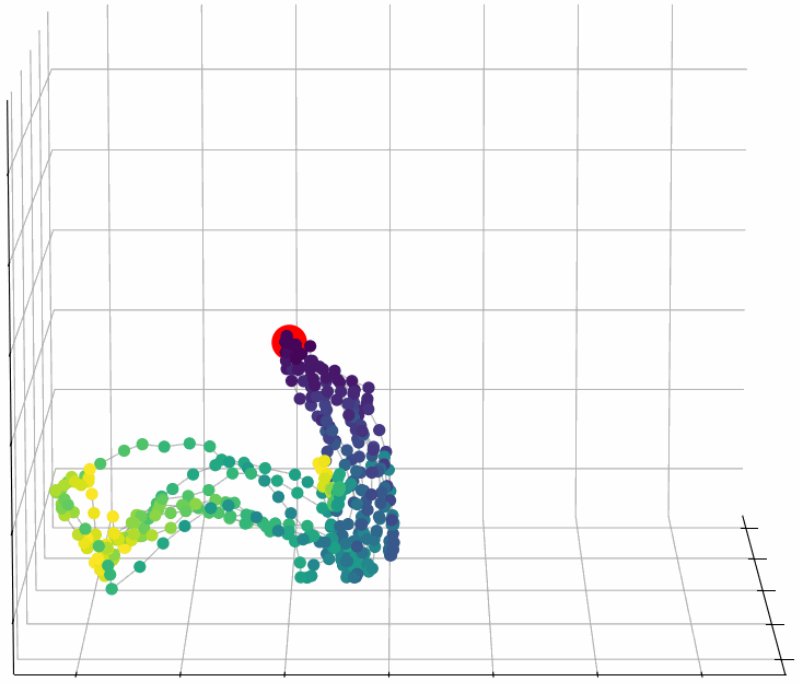
- Idea: train a noise predictor to steer diffusion robot policies



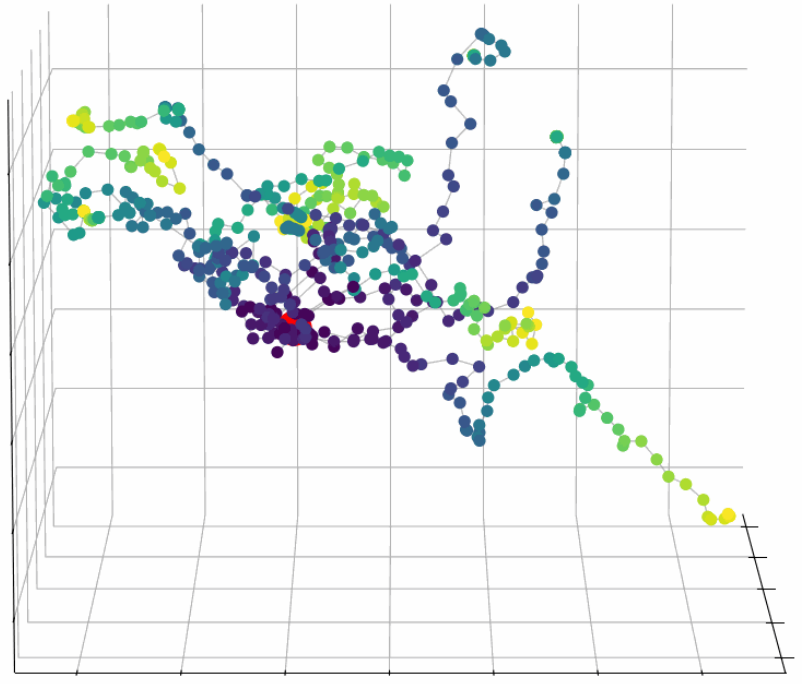
Steering Your Diffusion Policy with Latent Space Reinforcement Learning. Wagenmaker et al.

Idea 2: Learning New Tasks with Fixed Action Distribution

DSRL

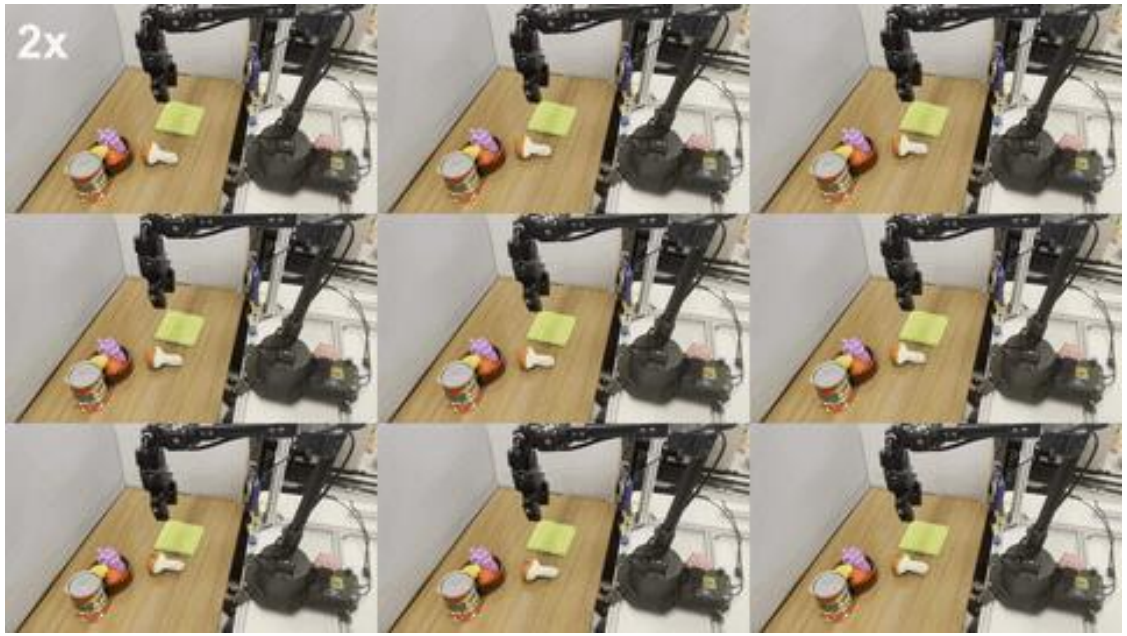


Vanilla RL

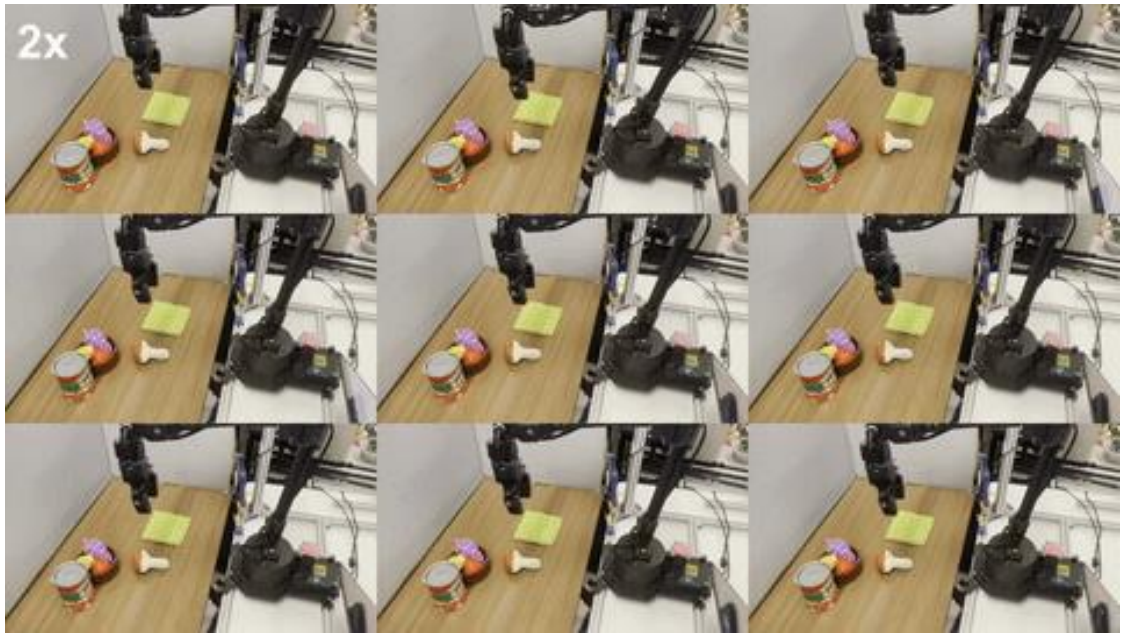


Idea 2: Learning New Tasks with Fixed Action Distribution

DSRL



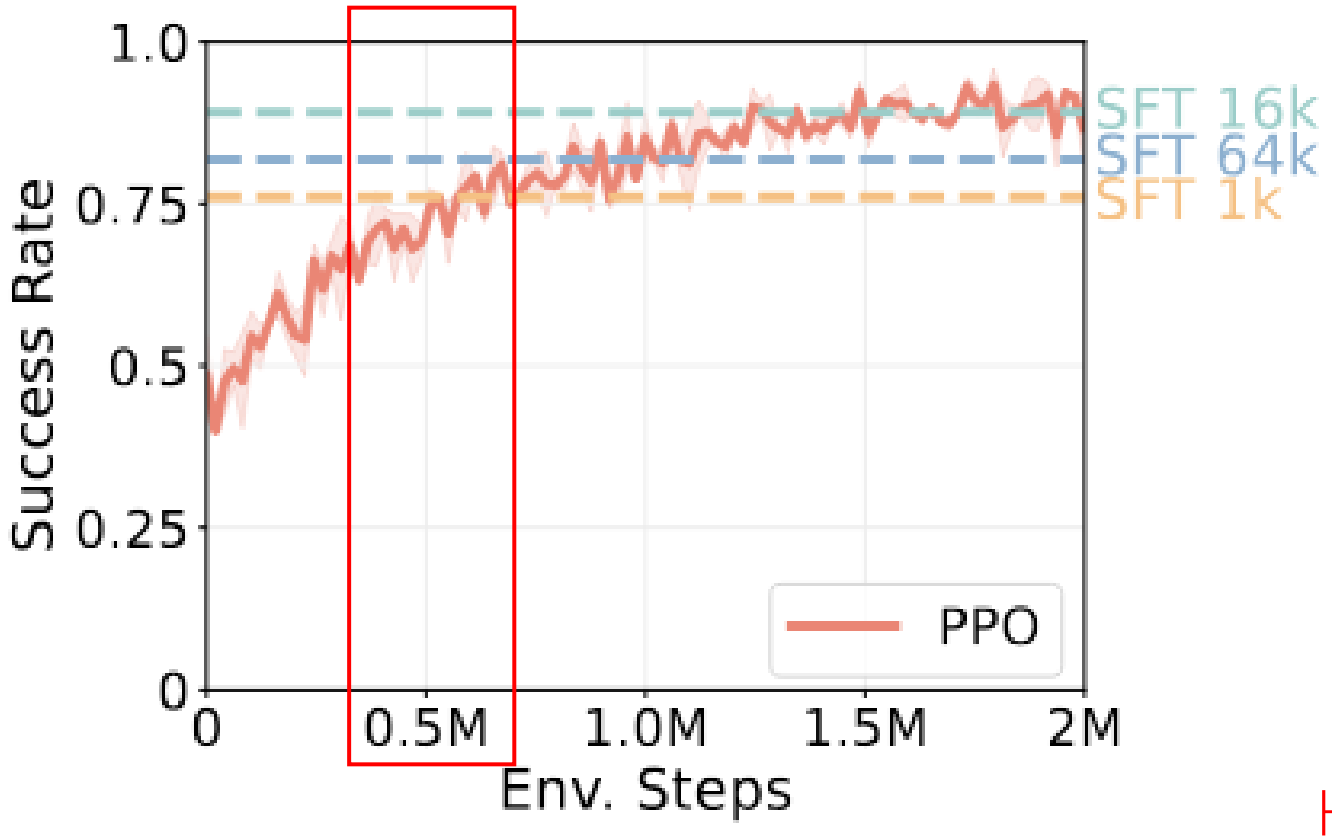
Vanilla RL



Problems of Fine-tuning VLAs via RL in the Real-World

- Heavy computational overhead of fine-tuning the entire VLA
- Catastrophic forgetting issue after fine-tuning
- Reinforcement learning is sample inefficient

Even Fine-tuning Strong Policies with RL is Sample Inefficient…

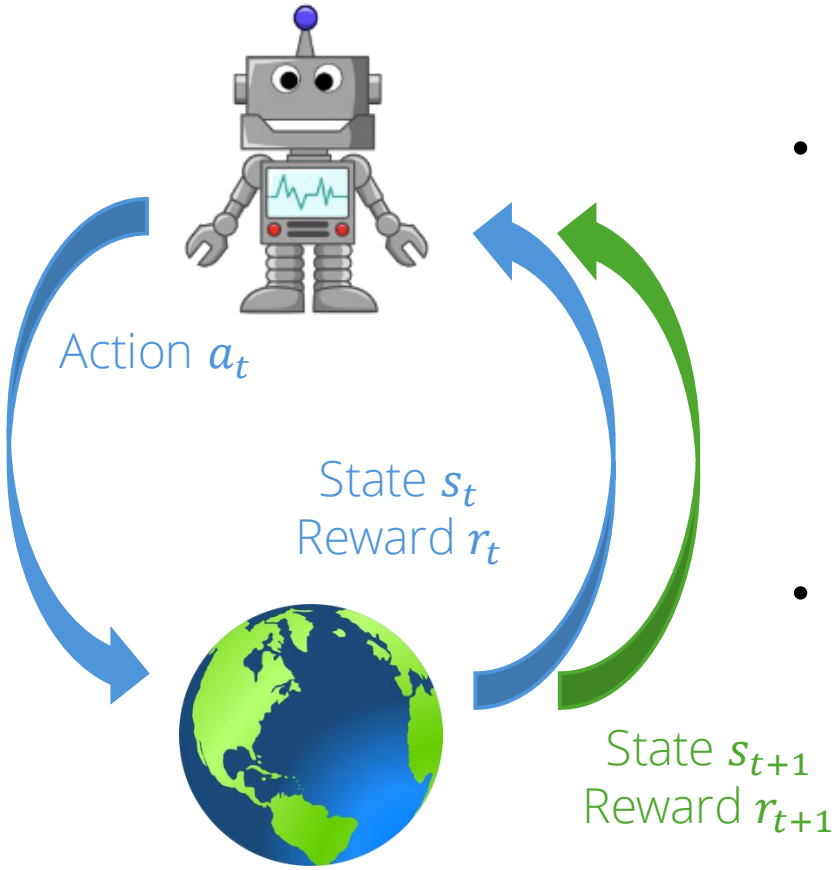


- It takes 0.5M env. Steps to reach ok-ish performance
- Assume the throughput of VLA as 10 FPS
- Fine-tuning a VLA for 0.5M env steps takes at least a day, if robot execution is instantaneous
- But robot execution takes time and we often need to learn multiple tasks in different environments….

How to enhance RL's sample efficiency?

What Can RL Bring to VLA Generalization? An Empirical Study. Liu et al.

Recap: Reinforcement Learning Objective



- A trajectory of interaction in the environment

$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

- Maximize the expected value of the cumulative sum of reward

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Offer more informative reward signals?

Slide adapted from S. Levine

Idea: Dense Reward Shaping with Human Feedback (RLHF)

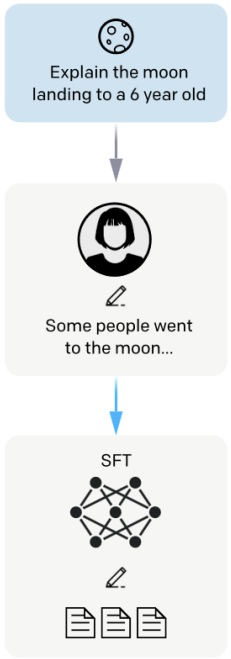
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



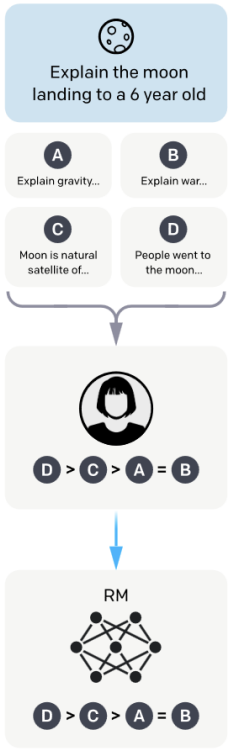
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

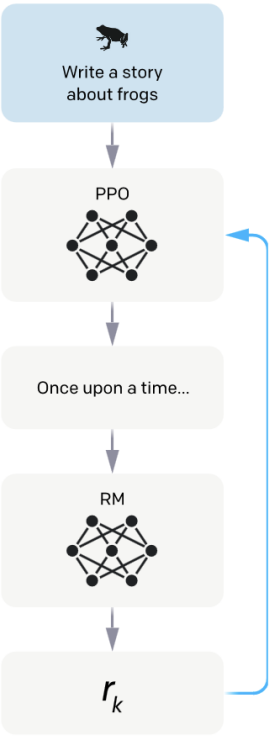
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

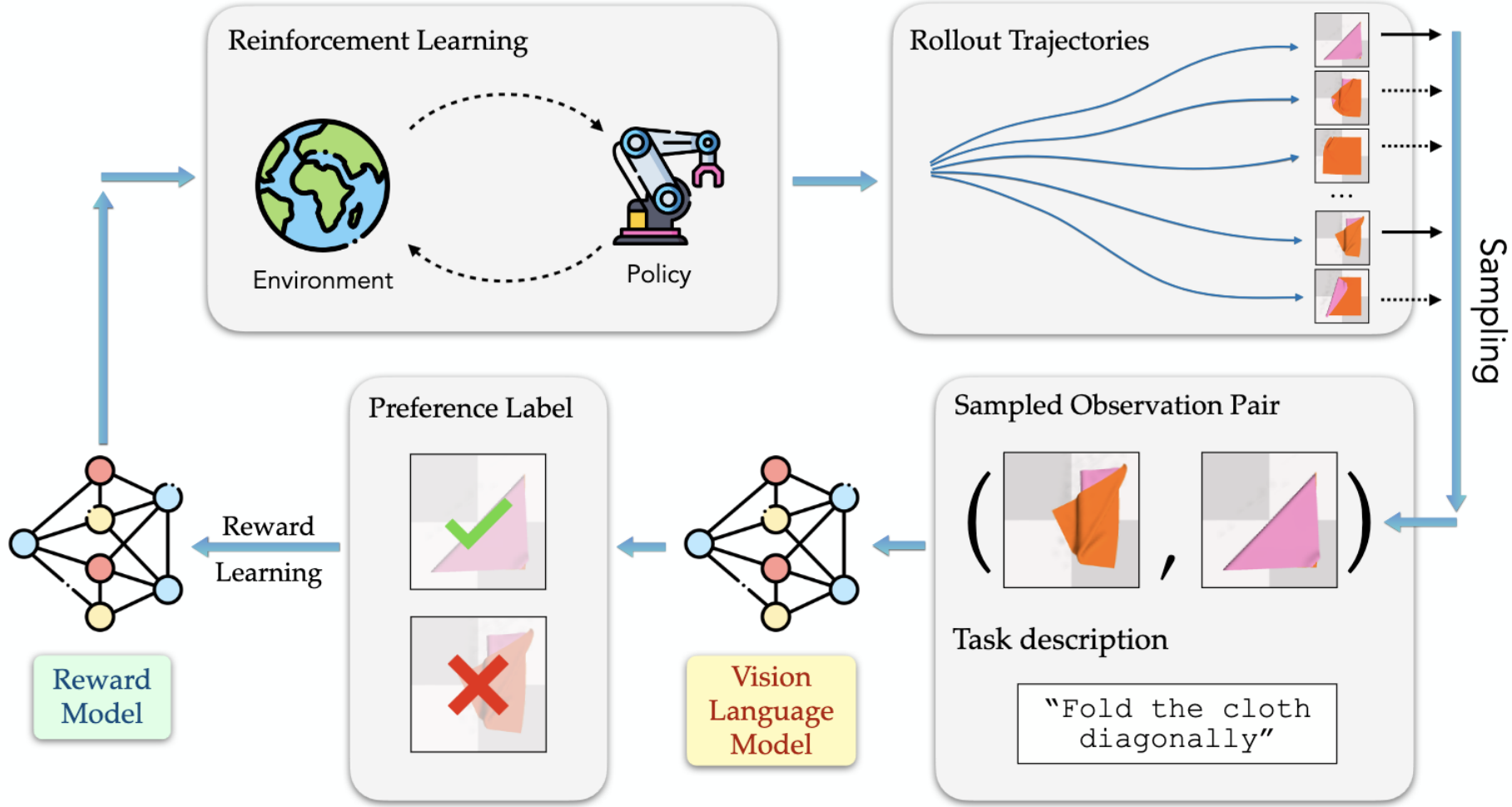
The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



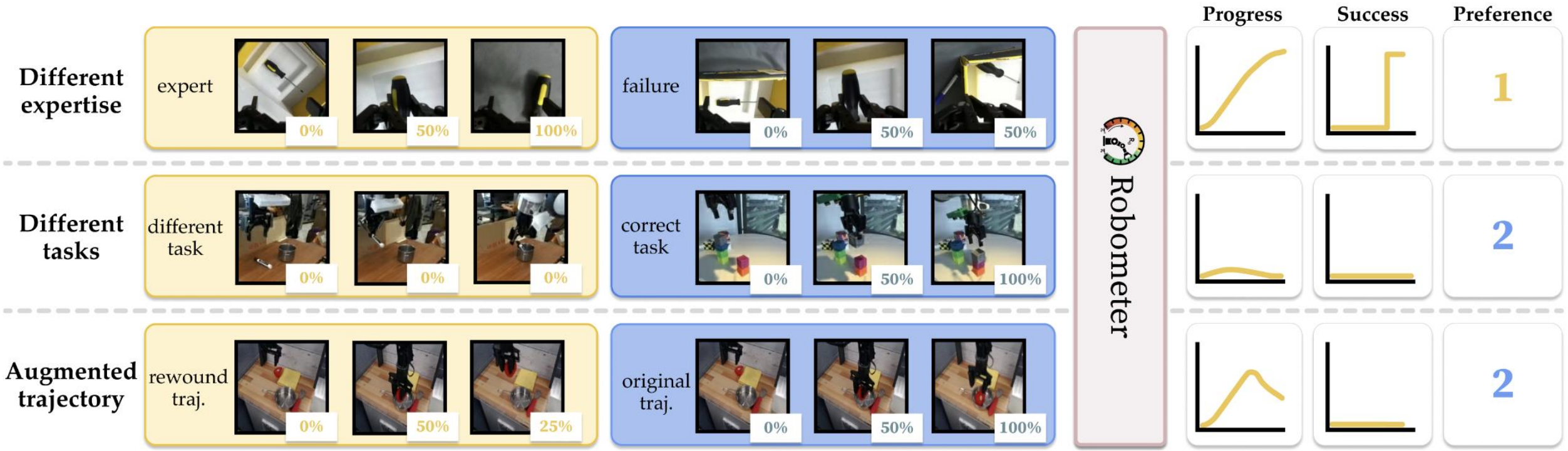
Training language models to follow instructions with human feedback. Ouyang et al.

Idea: Dense Reward Shaping with VLM Feedback



RL-VLM-F: Reinforcement Learning from Vision Language Foundation Model Feedback. Wang et al.

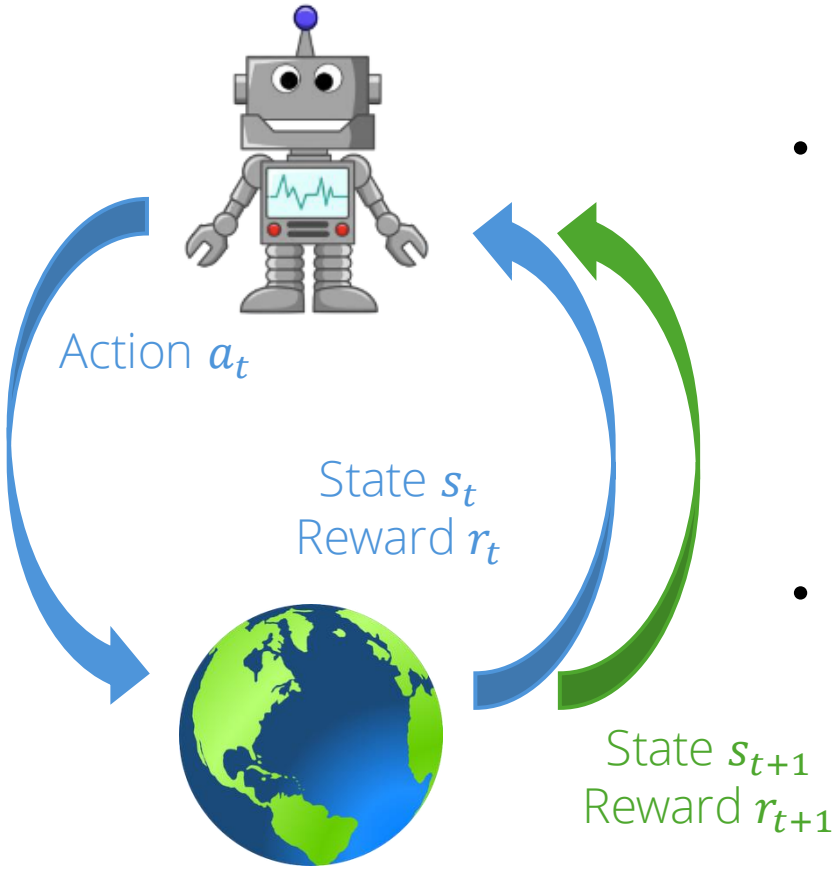
Idea: Dense Reward Shaping with Reward Model



We'll cover more in future lectures!

ROBOMETER: Scaling General-purpose Robotic Reward Model via Trajectory Comparisons. Liang et al.

Recap: Reinforcement Learning Objective



- A trajectory of interaction in the environment

$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

- Maximize the expected value of the cumulative sum of reward

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Collect more informative trajectories?

Offer more informative reward signals?

Slide adapted from S. Levine

Idea: DAGGER?

Initialize $\mathcal{D} \leftarrow \emptyset$.
Initialize $\hat{\pi}_1$ to any policy in Π.
for $i = 1$ **to** N **do**
  Let $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$.
  Sample T-step trajectories using π_i.
  Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by expert.
  Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{D}_i$.
  Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$.
end for
Return best $\hat{\pi}_i$ on validation.

Algorithm 3.1: DAGGER Algorithm.

New dataset:

$o_1^1, a_1^1, o_2^1, a_2^1, \ldots, o_T^1, a_T^1$
$o_1^2, a_1^2, o_2^2, a_2^2, \ldots, o_T^2, a_T^2$
...
$o_1^N, a_1^N, o_2^N, a_2^N, \ldots, o_T^N, a_T^N$

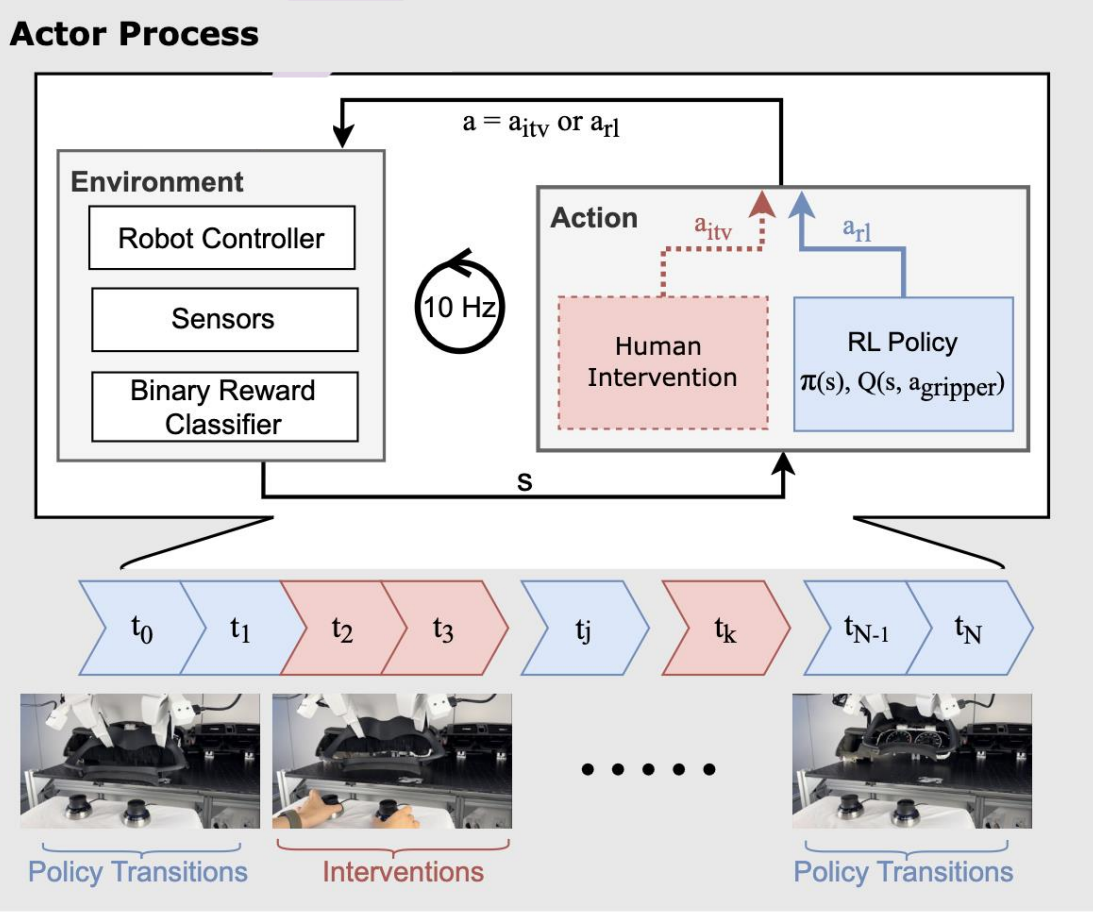
+

$o_1^S, a_1^S, o_2^S, a_2^S, \ldots, o_T^S, a_T^S$
$o_1^U, a_1^U, o_2^U, a_2^U, \ldots, o_T^U, a_T^U$
...
$o_1^V, a_1^V, o_2^V, a_2^V \ldots, o_T^V, a_T^V$

But this is still imitation learning!

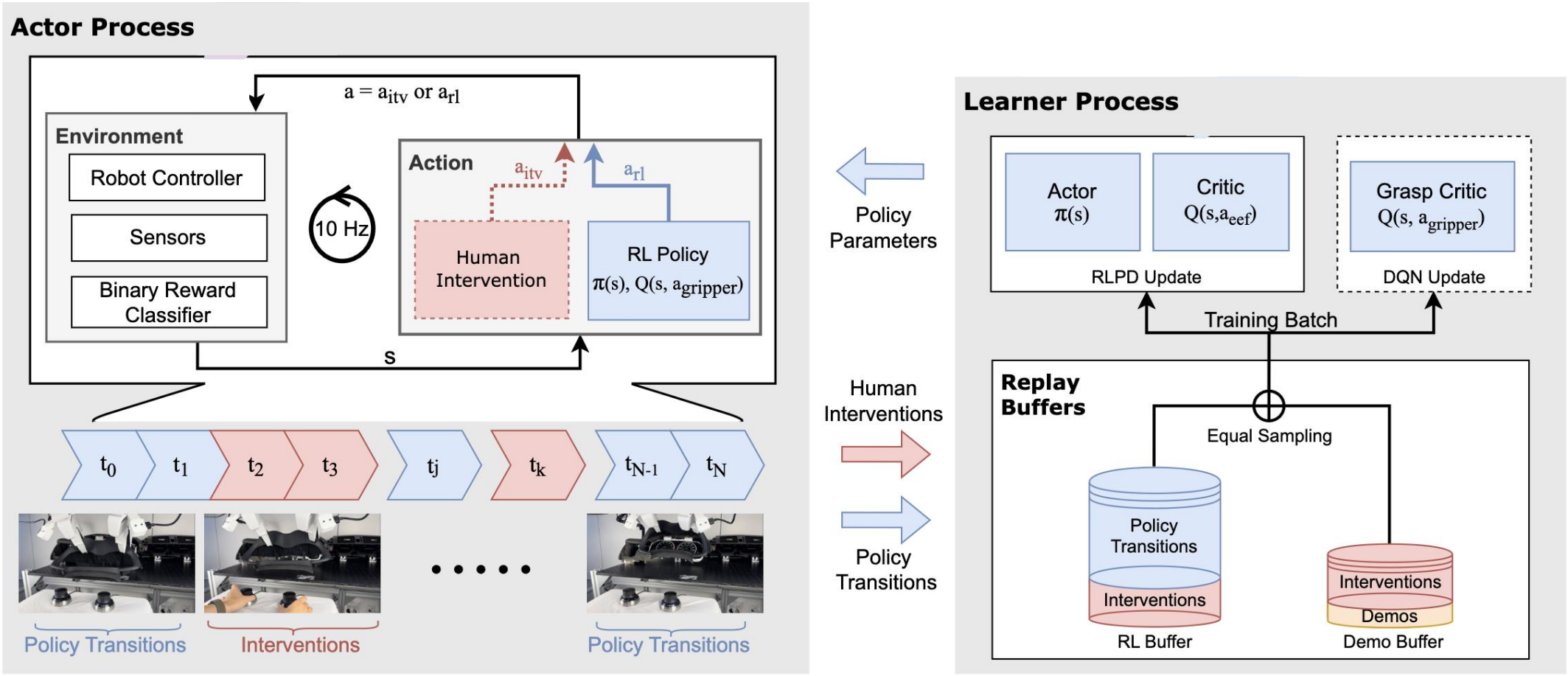
A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, Ross et al.

Idea: Collect Trajectories with Experts but Train Policies with RL



Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning. Luo et al.

Idea: Collect Trajectories for Correction with Experts but Train Policies with RL (Put Human in the Loop)



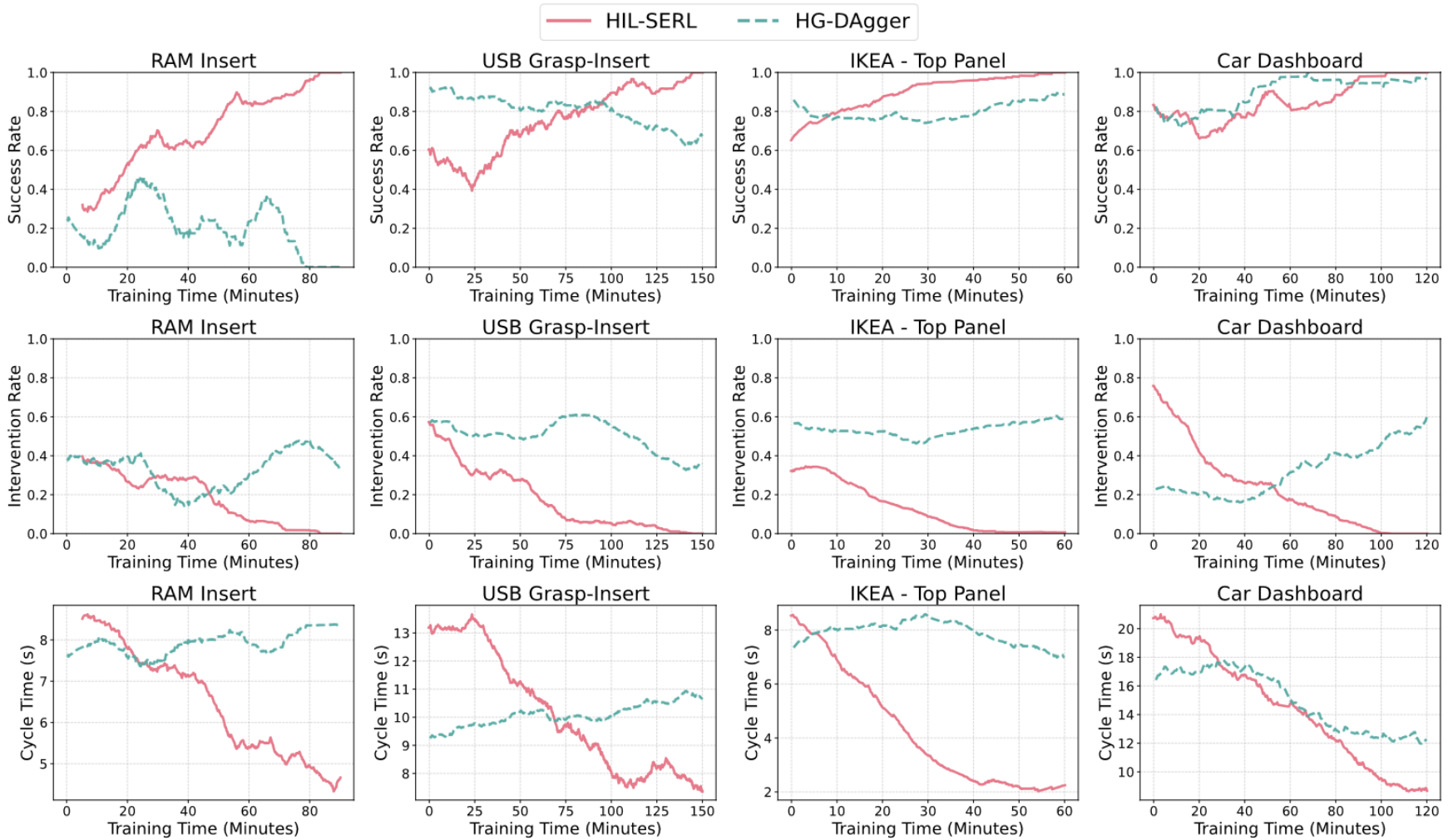
Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning. Luo et al.

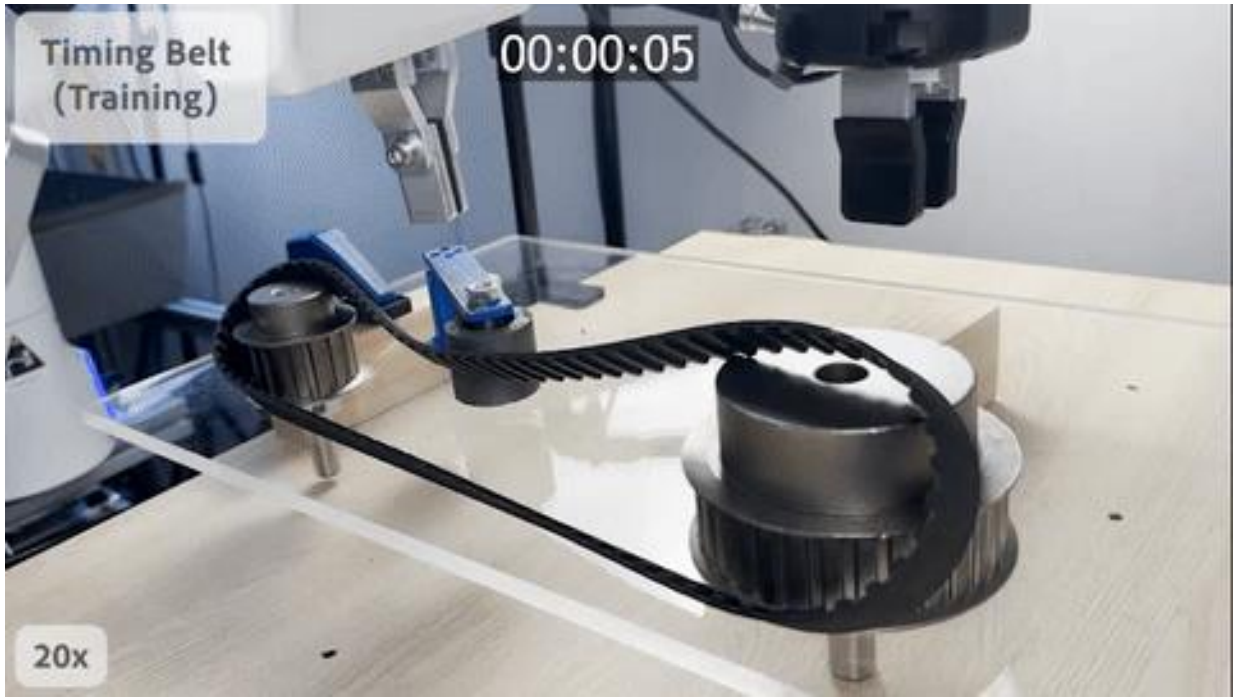
Result: Learn Faster than Imitation Learning

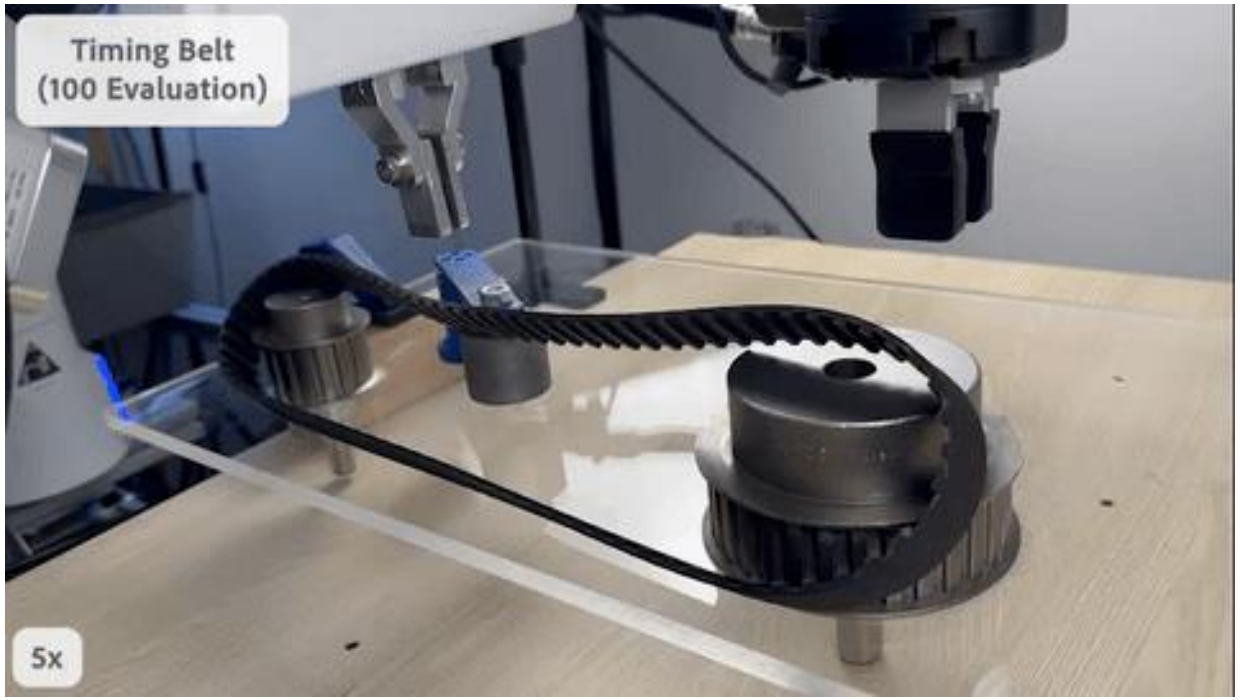
Task	Training Time (h)	Success Rate (%)		Cycle Time (s)	
		BC	HIL-SERL (ours)	BC	HIL-SERL (ours)
RAM Insertion	1.5	29	**100** (+245%)	8.3	**4.8** (1.7x faster)
SSD Assembly	1	79	**100** (+27%)	6.7	**3.3** (2x faster)
USB Grasp-Insertion	2.5	26	**100** (+285%)	13.4	**6.7** (2x faster)
Cable Clipping	1.25	95	**100** (+5%)	7.2	**4.2** (1.7x faster)
IKEA - Side Panel 1	2	77	**100** (+30%)	6.5	**2.7** (2.4x faster)
IKEA - Side Panel 2	1.75	79	**100** (+27%)	5.0	**2.4** (2.1x faster)
IKEA - Top Panel	1	35	**100** (+186%)	8.9	**2.4** (3.7x faster)
IKEA - Whole Assembly	–	1/10	**10/10** (+900%)	–	–
Car Dashboard Assembly	2	41	**100** (+144%)	20.3	**8.8** (2.3x faster)
Object Handover	2.5	79	**100** (+27%)	16.1	**13.6** (1.2x faster)
Timing Belt Assembly	6	2	**100** (+4900%)	9.1	**7.2** (1.3x faster)
Jenga Whipping	1.25	8	**100** (+1150%)	–	–
Object Flipping	1	46	**100** (+117%)	3.9	**3.8** (1.03x faster)
Average	–	49.7	**100** (+101%)	9.6	**5.4** (1.8x faster)

Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning. Luo et al.

Result: Learn Faster than DAGGER







Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning. Luo et al.

Problems of Fine-tuning VLAs via RL in the Real-World

- Heavy computational overhead of fine-tuning the entire VLA
- Catastrophic forgetting issue after fine-tuning
- Reinforcement learning is sample inefficient
- Scene reset is labor intensive

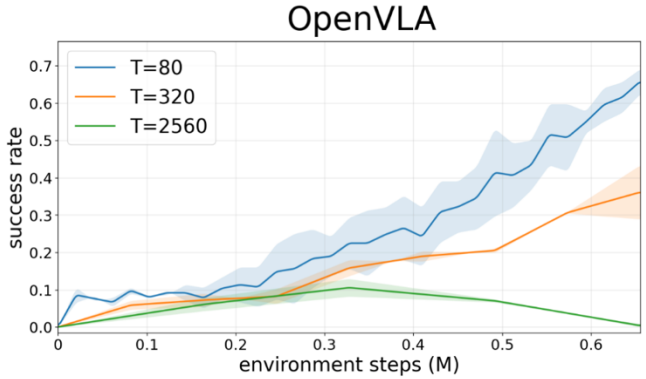
Let's See Another Example of Real-World RL

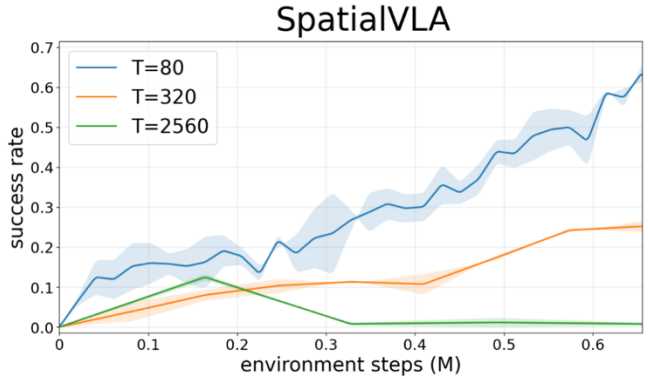


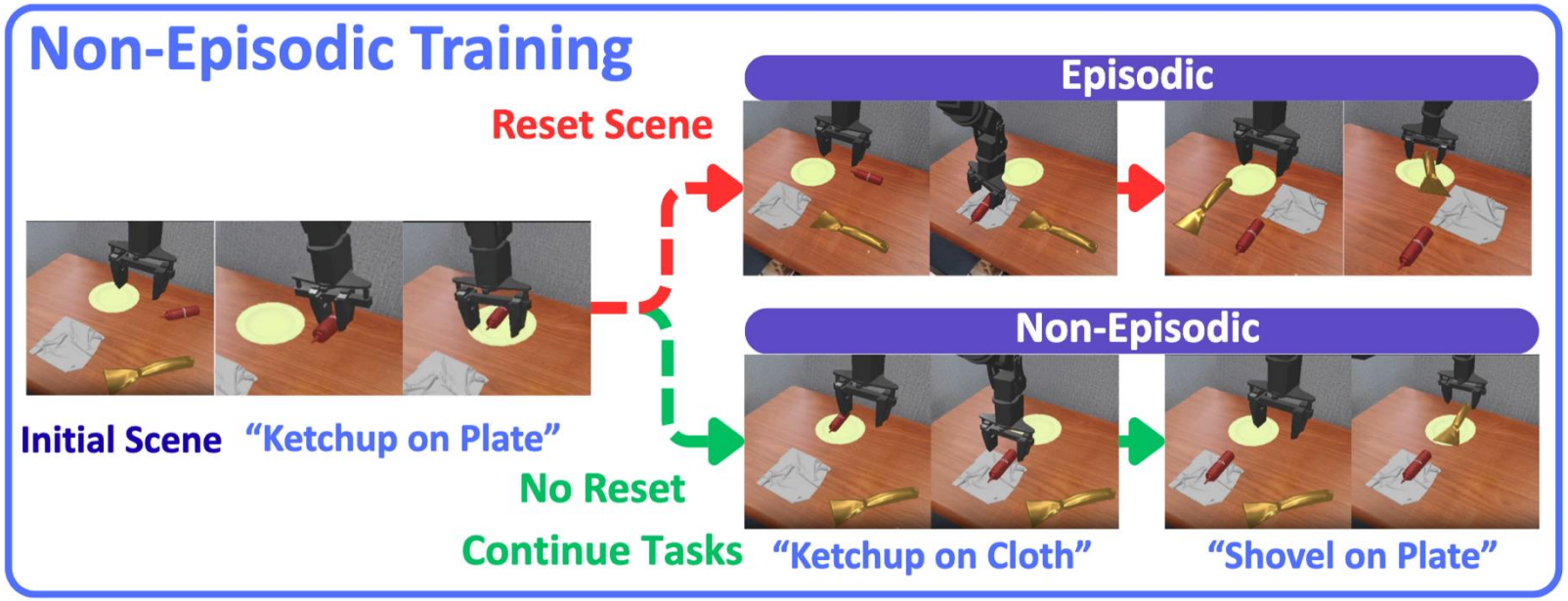
Scene reset is the most time-consuming…

https://youtu.be/QZvu8M02BeE?si=XmlK6vhVGDCsyhf0

Why Not Reducing the Frequency of Scene Reset?



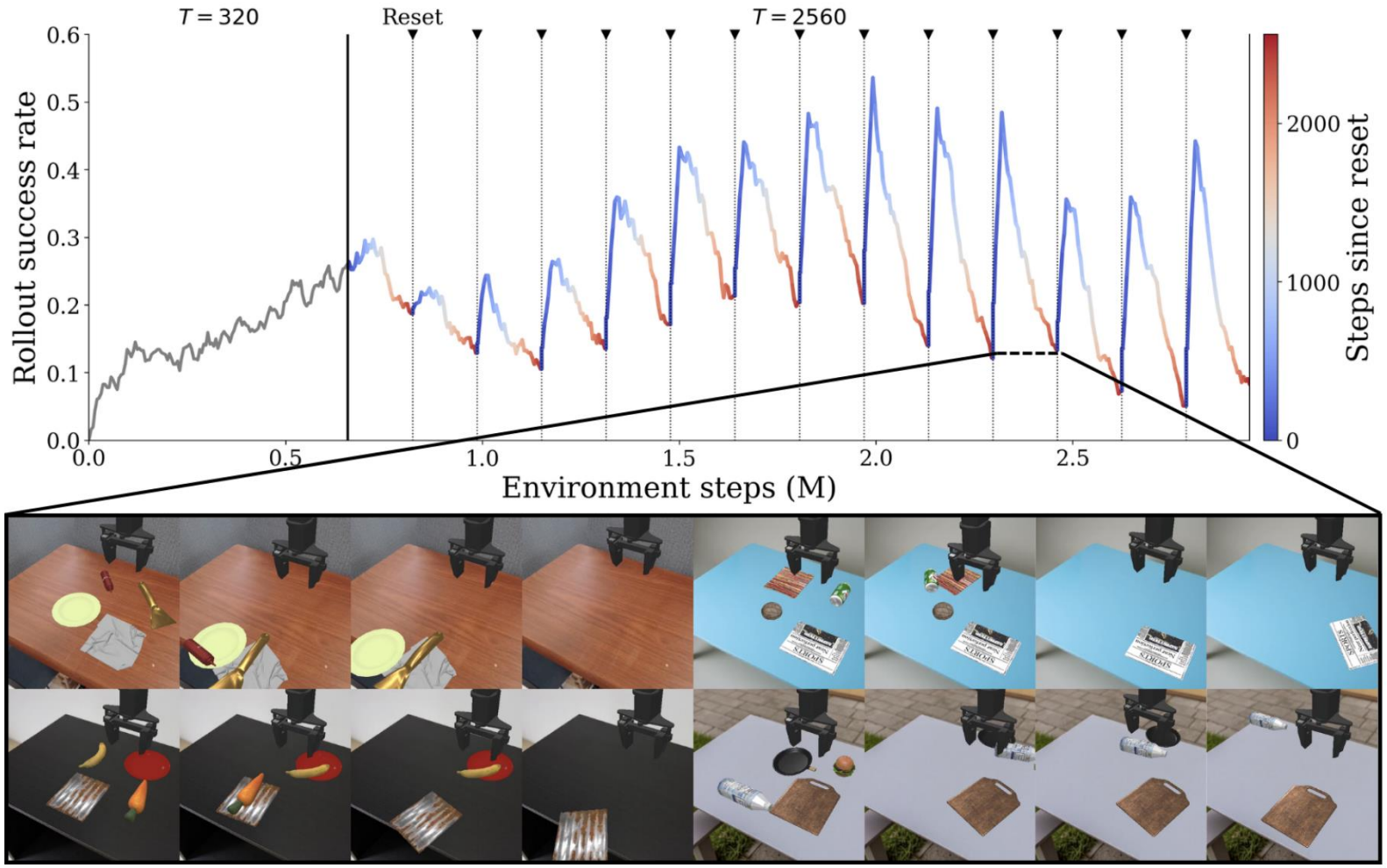




Reducing reset frequency doesn't work...

CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

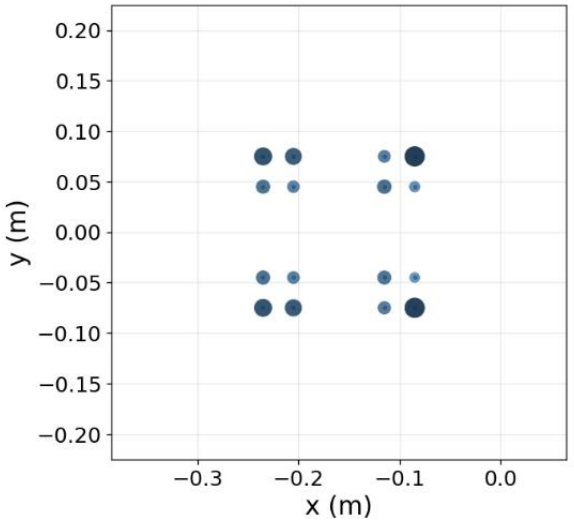
Rollout Success Decays between Scene Resets



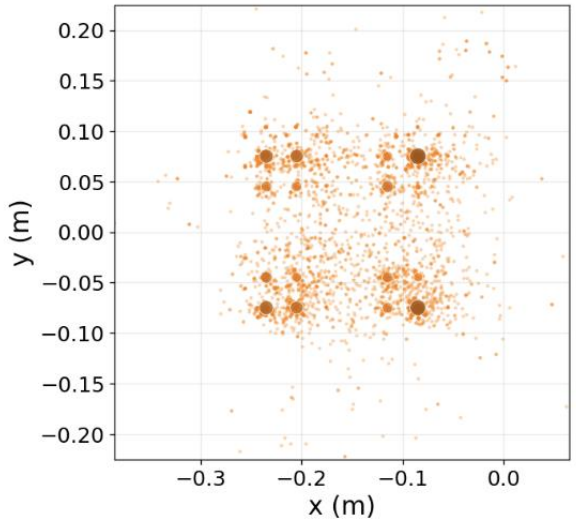
CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

Why Non-Episodic RL is Hard?

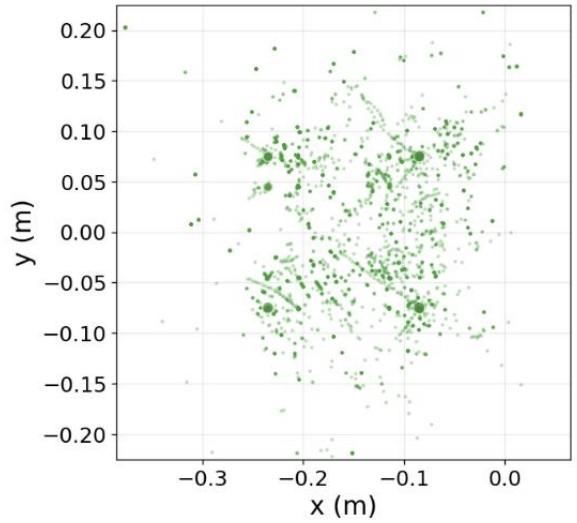
- Scene configurations are more diverse with less frequent scene reset



(a) $T = 80$



(b) $T = 320$

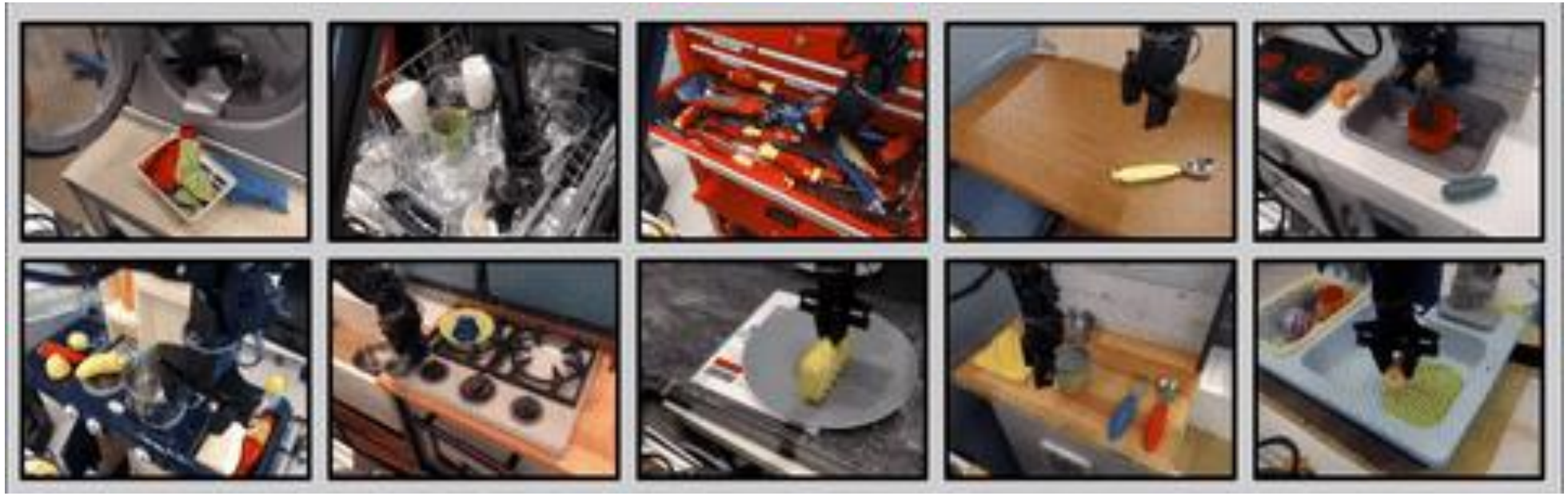


(c) $T = 2560$

CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

Why Non-Episodic RL is Hard?

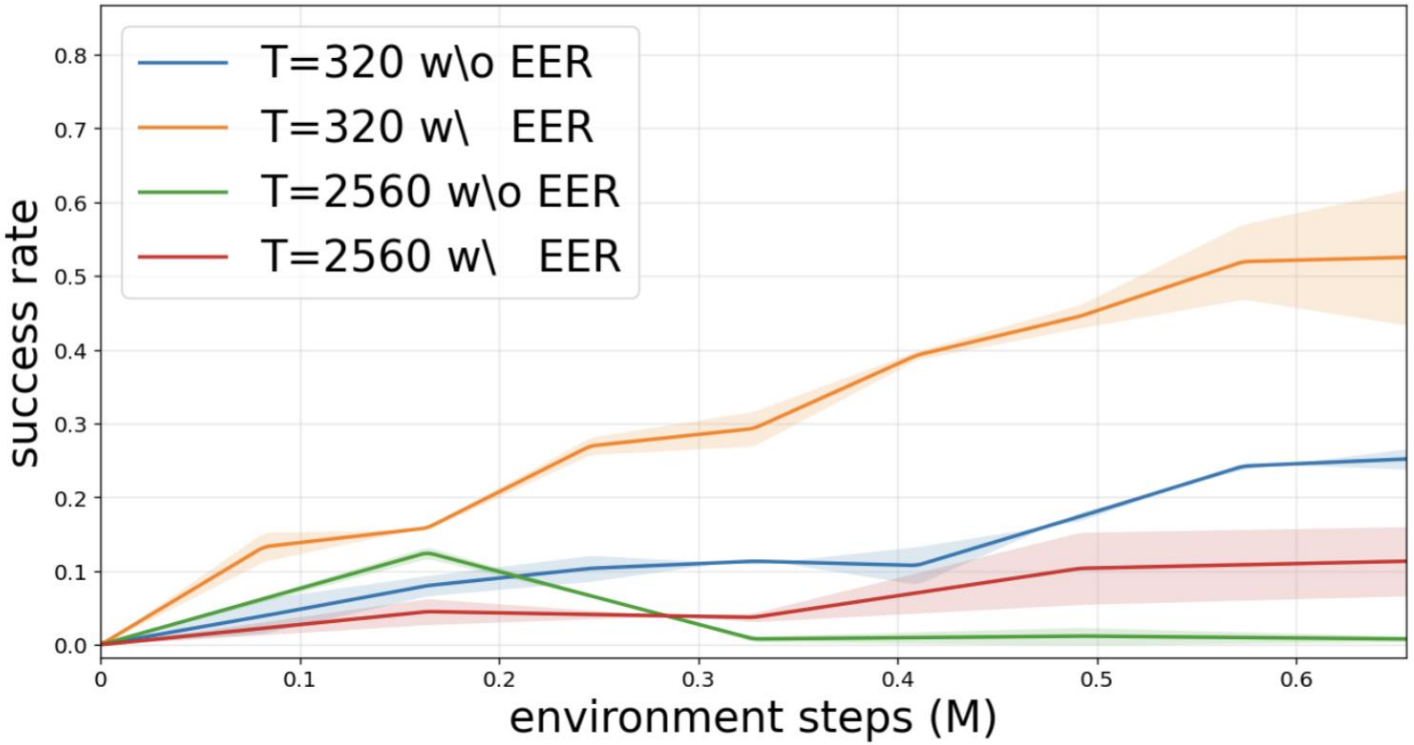
- Expert demonstrations for pre-training VLAs have strong bias in initial end-effector poses



Video source: Bridge.v2 dataset

Why Non-Episodic RL is Hard?

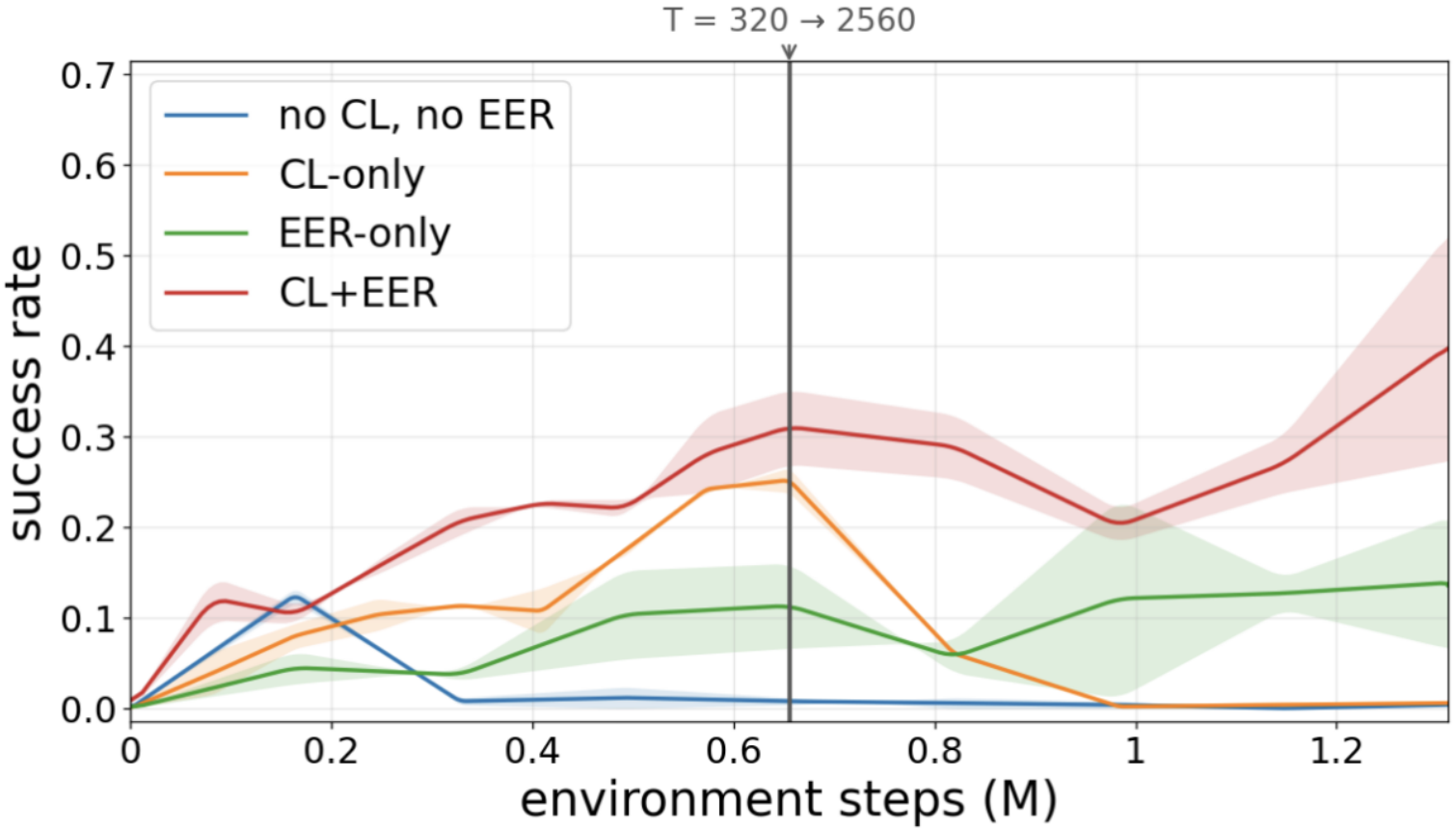
- Expert demonstrations for pre-training VLAs have strong bias in initial end-effector poses



CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

Why Non-Episodic RL is Hard?

- The strategy to allocate reset budget is sub-optimal.

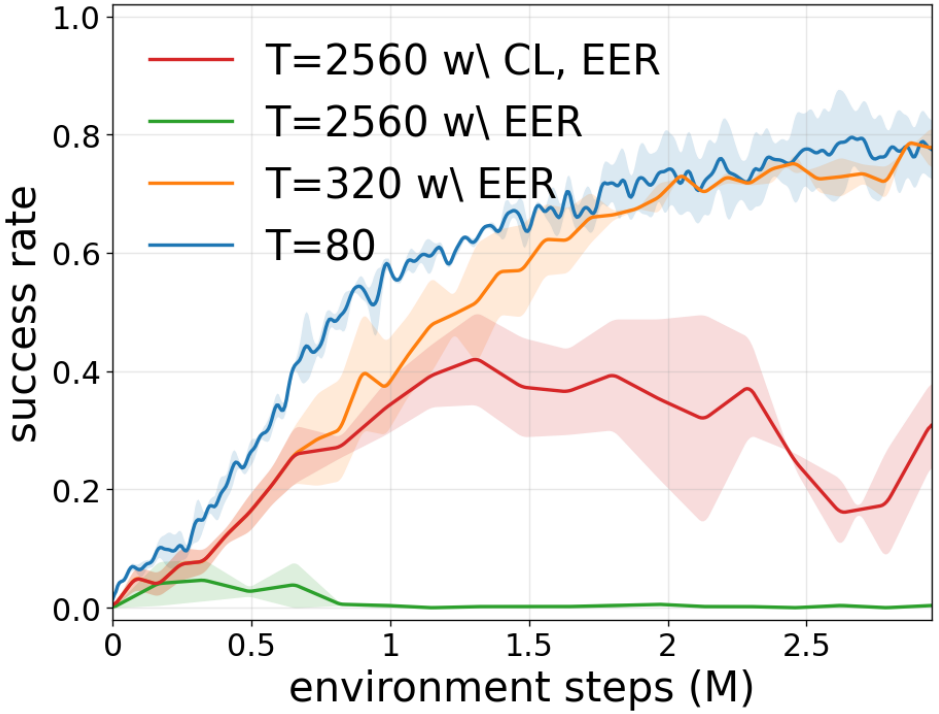


- Even naïve curriculum learning (CL) strategy enhances non-episodic RL:
  - First train with frequent scene reset (T=320)
  - Next train with sparse scene reset (T=2560)

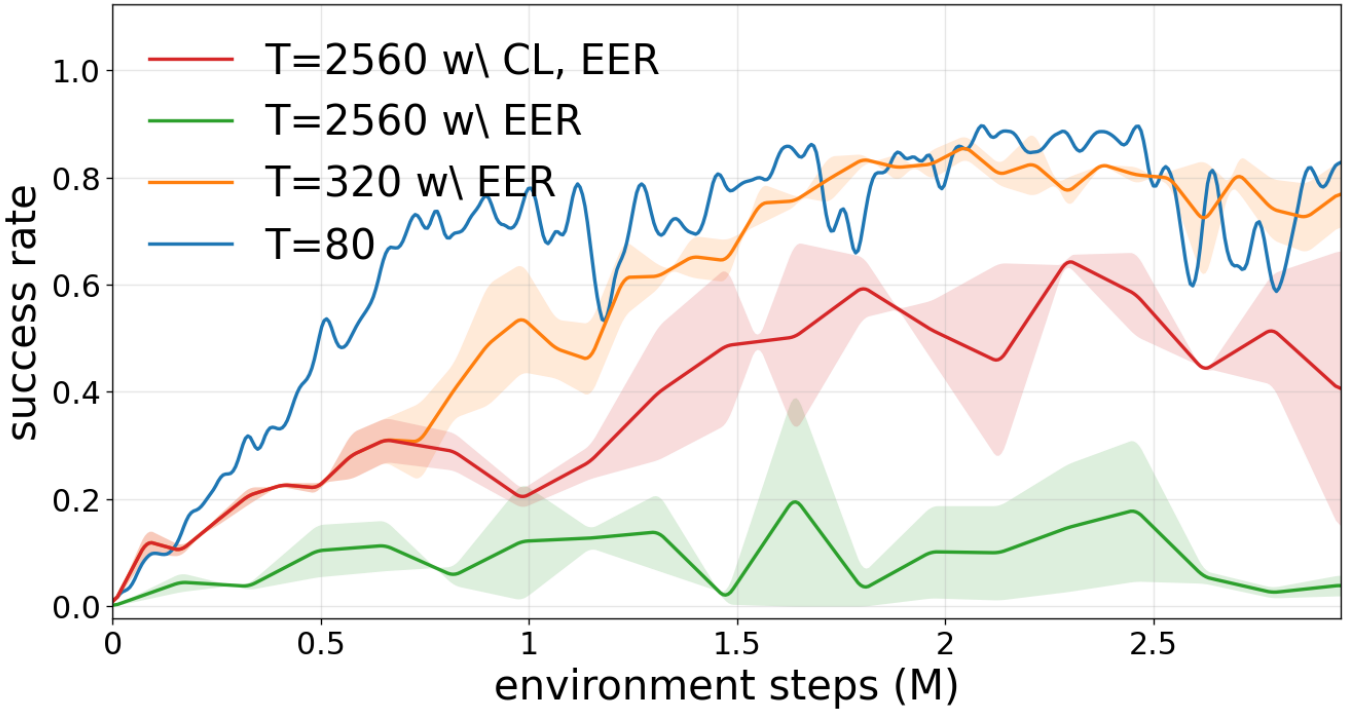
CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

Unfortunately, Nothing Works in Non-Episodic RL

OpenVLA



SpatialVLA



CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

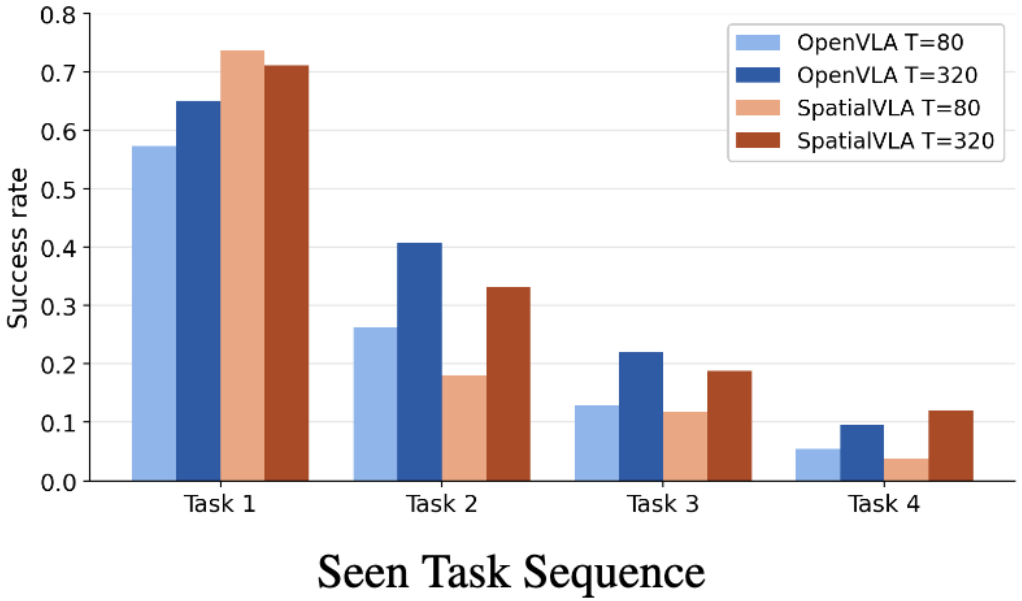
But Non-Episodic RL Enhances a Policy's Robustness to Unseen Scene Configurations



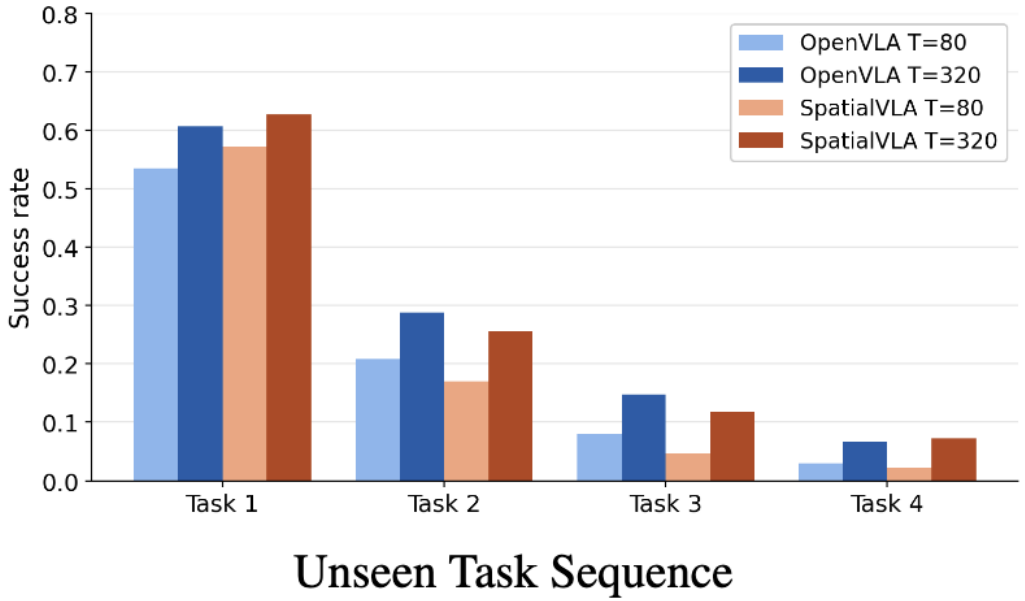
Model	T	Overlap	Cluttered
OpenVLA	80	0.554	0.494
	320	**0.593**	**0.536**
SpatialVLA	80	0.552	0.512
	320	**0.643**	**0.550**

CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

But Non-Episodic RL Enhances a Policy's Robustness to Unseen Task Sequences



Seen Task Sequence



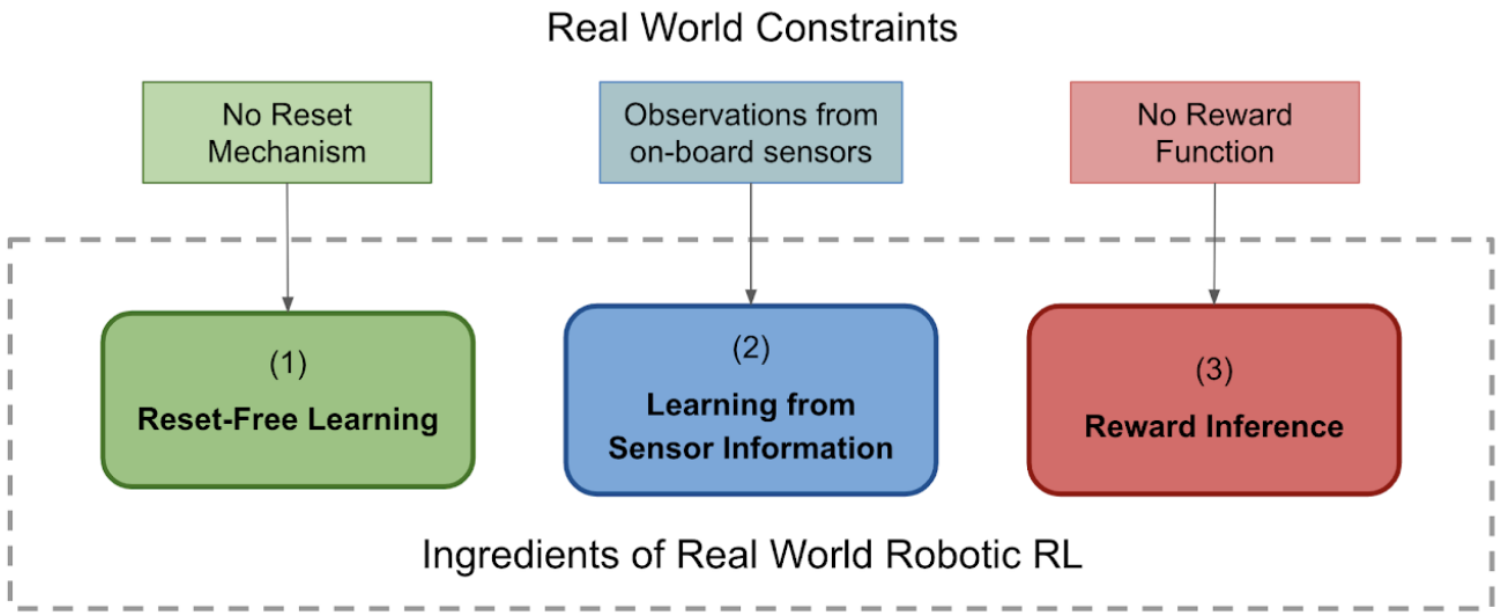
Unseen Task Sequence

CRONOS: Benchmarking Non-Episodic Reinforcement Learning for Robotic Manipulation. Wu et al.

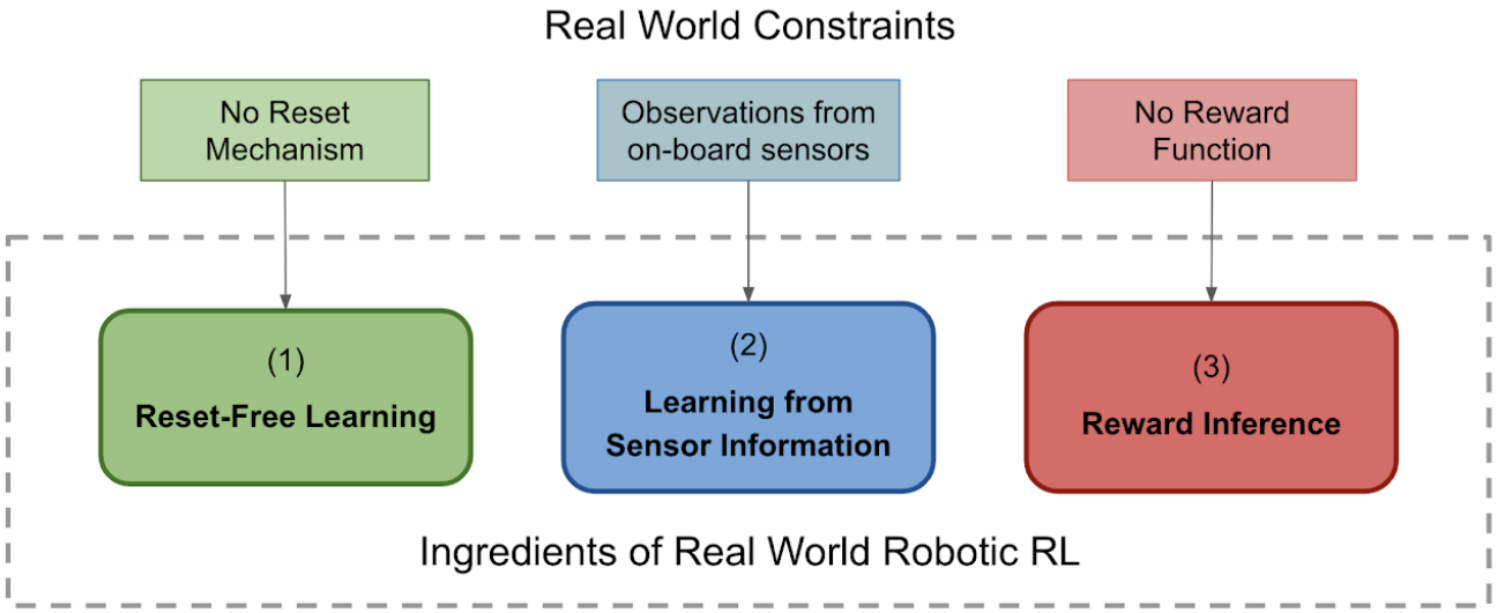
What are Other Challenges of Real-World RL?

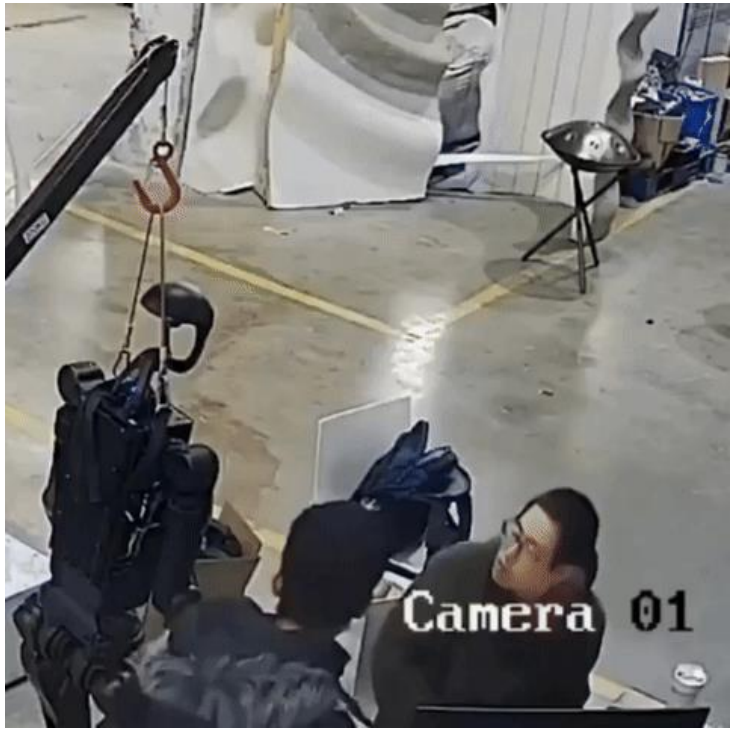
THE INGREDIENTS OF REAL-WORLD ROBOTIC REINFORCEMENT LEARNING

Henry Zhu[*1], **Justin Yu**[*1], **Abhishek Gupta**[*1], **Dhruv Shah**[1],
Kristian Hartikainen[2], **Avi Singh**[1], **Vikash Kumar**[3], **Sergey Levine**[1]
[1] University of California, Berkeley [2] University of Oxford [3] University of Washington



Don’t Forget Safety! But How?





https://youtube.com/shorts/RKUeRVTxzjY?si=BpQ7Fez72DkKfUDf